Université de Bretagne Occidentale
Faculté des Sciences et Techniques
Département Informatique

Mémoire de fin d'études

Pour l'obtention du diplôme de Master en Informatique

Parcours : Logiciels pour les Systèmes Embarqués

Déploiement de LLM multi-tenant en périphérie

Réalisé par :

Sadi Lina

Encadré par :

M. Boukhobza Jalil
M. Lannurien Vincent
Mme Bouzouad Meriem

Organisme d'accueil :

École Nationale Supérieure de Techniques Avancées, Brest, France

2025 / 2026

Remerciements

Tout d'abord, je tiens à exprimer ma profonde gratitude à mon promoteur, **M. Jalil BOUKHOBZA**, pour son accompagnement constant, sa patience, son engagement et la confiance qu'il m'a témoignée tout au long de ce travail.

Mes remerciements s'adressent également à **Mme Meriem BOUZOUAD**, pour sa disponibilité, son accompagnement tout au long du projet, ses conseils avisés et son accueil bienveillant. Je tiens aussi à exprimer toute ma reconnaissance à **M. Vincent LANNURIEN**, pour son engagement constant, sa disponibilité et son accompagnement dans ce travail.

J'adresse également mes remerciements à **M. Stéphane RUBINI**, tuteur de l'UBO, pour son encadrement attentif, son professionnalisme et son soutien.

Je tiens également à remercier **Chakib**, avec qui j'ai eu le plaisir de partager cette aventure à l'UBO et à ENSTA Bretagne. Son aide, nos échanges et notre collaboration tout au long du stage ont largement contribué à l'avancement de ce travail.

J'exprime ma reconnaissance au **jury**, qui me fera l'honneur d'évaluer ce travail, ainsi qu'à l'**équipe pédagogique et administrative de l'UBO** pour ses efforts constants visant à offrir une formation de qualité.

Enfin, j'adresse ma gratitude la plus sincère à ma **famille** pour son soutien indéfectible, véritable source de motivation à chaque étape de ce parcours. Je remercie également mes **amis**, ainsi que toutes celles et ceux qui, de près ou de loin, ont contribué à la réalisation de ce mémoire.

Résumé

L'essor des modèles de langage de grande taille (LLM) et la multiplication de leurs usages conduisent à envisager leur déploiement au plus près des utilisateurs, notamment sur des plateformes Edge. Ces environnements offrent des avantages en termes de latence, de confidentialité et de dépendance au cloud, mais disposent de ressources de calcul et de mémoire plus contraintes. Ces limitations deviennent particulièrement importantes dans un contexte multi-utilisateur, où plusieurs requêtes doivent partager les mêmes ressources. Si de nombreux moteurs et mécanismes d'optimisation ont été développés pour l'inférence des LLM, leur comportement dans de telles conditions reste encore principalement étudié sur des infrastructures serveur, tandis que les travaux Edge se concentrent davantage sur le modèle, la quantification, la charge ou la plateforme.

Dans ce contexte, ce mémoire propose une caractérisation expérimentale de l'inférence LLM multi-utilisateur sur des plateformes NVIDIA Jetson. À partir de l'état de l'art, deux moteurs reposant sur des choix de conception différents, `llama.cpp` et vLLM, ont été retenus et comparés selon cinq axes : charge de travail, concurrence, partage des ressources, modèle, quantification et plateforme. Les expériences, menées principalement sur une Jetson AGX Orin puis complétées sur une Jetson Orin Nano, évaluent conjointement les performances côté utilisateur à travers le TTFT, la latence, le TPOT et le débit, ainsi que l'utilisation du CPU, du GPU et de la mémoire. Les résultats montrent que vLLM offre les meilleures performances dans la majorité des configurations étudiées, particulièrement lorsque la charge ou la concurrence augmente, mais mobilise davantage de CPU et de mémoire. À l'inverse, `llama.cpp` présente une empreinte système plus légère et se dégrade proportionnellement moins lorsque les ressources hôtes sont partagées. L'étude montre également que plusieurs tendances observées sur l'AGX Orin se maintiennent sur l'Orin Nano. Ces résultats mettent ainsi en évidence un compromis entre performances et ressources mobilisées dans le choix d'un moteur d'inférence pour un environnement Edge multi-utilisateur.

Mots-clés : modèles de langage, LLM, Edge computing, inférence multi-utilisateur, moteurs d'inférence, llama.cpp, vLLM, NVIDIA Jetson, performances, ressources système

---

Table des matières

Remerciements **i**

Résumé **ii**

Acronymes **vii**

Introduction générale **1**

1 Inférence des LLM sur *Edge* **4**
1.1 Introduction 4
1.2 Principes de l'inférence des LLM 4
1.2.1 Architecture des LLM 4
1.2.2 Processus d'inférence 6
1.3 Inférence multi-utilisateur 8
1.3.1 Ordonnancement des requêtes 8
1.3.2 Gestion de la mémoire 10
1.4 Adaptation des modèles aux plateformes *Edge* 11
1.5 Conclusion 12

2 État de l'art **13**
2.1 Introduction 13
2.2 Systèmes et optimisations pour l'inférence des *Large Language Models* (LLM) 13
2.2.1 Solutions adaptées au contexte multi-utilisateur 13
2.2.2 Solutions adaptées aux plateformes *Edge* 14
2.2.3 Synthèse 15
2.3 Caractérisation des systèmes d'inférence 15
2.3.1 Caractérisation en contexte multi-utilisateur 16
2.3.2 Caractérisation sur les plateformes *Edge* 16
2.3.3 Synthèse et positionnement 17
2.4 Conclusion 18

3 Méthodologie expérimentale **19**
3.1 Introduction 19
3.2 Questions de recherche 19
3.3 Cadre expérimental commun 20
3.3.1 Plateformes expérimentales 20
3.3.2 Configurations des moteurs 20
3.3.3 Métriques et instrumentation 22
3.3.4 Protocole expérimental 24

3.4 Conclusion 25

4 Campagnes expérimentales, résultats et analyse 26
4.1 Introduction 26
4.2 Influence de la charge de travail 26
4.2.1 Configuration de l'expérience 26
4.2.2 Résultats et analyse 27
4.3 Influence de la concurrence 29
4.3.1 Configuration de l'expérience 30
4.3.2 Résultats et analyse 30
4.4 Influence du partage des ressources 33
4.4.1 Configuration de l'expérience 33
4.4.2 Résultats et analyse 34
4.5 Généralisation à d'autres modèles et quantifications 36
4.5.1 Configuration de l'expérience 36
4.5.2 Résultats et analyse 37
4.6 Généralisation à une autre plateforme *Edge* 40
4.6.1 Configuration de l'expérience 40
4.6.2 Résultats et analyse 41
4.7 Synthèse du chapitre 45
4.7.1 Synthèse des résultats 45
4.7.2 Choix du moteur selon la charge et les ressources disponibles 46
4.8 Conclusion 47

Conclusion générale 48

Bibliographie 50

Table des figures

1.1 Architecture simplifiée d'un Transformer de type *decoder-only*, adaptée de [21]. 5
1.2 Processus d'inférence autorégressive d'un LLM, adapté de [21]. 7
1.3 Comparaison des stratégies de *batching* statique, dynamique et continu. . . 9
1.4 Fragmentation interne et externe lors d'une allocation contiguë des caches *Key-Value* (KV) de plusieurs requêtes, adaptée de [13]. 11

4.1 Influence des longueurs d'entrée et de sortie sur les performances côté client à $C = 1$. 27
4.2 Influence des longueurs d'entrée et de sortie sur les ressources mobilisées et l'énergie dynamique à $C = 1$. 29
4.3 Influence de la concurrence sur les performances côté client pour $P128/O1024$. 30
4.4 Influence de la concurrence sur les ressources mobilisées et l'énergie dynamique pour $P128/O1024$. 32
4.5 Activité GPU de `llama.cpp` et vLLM sous concurrence pour $P128/O1024$. 33
4.6 Influence du partage des ressources sur les performances côté client pour $P128/O1024$ à $C = 4$. 34
4.7 Influence du partage des ressources sur les ressources mobilisées et l'énergie dynamique pour $P128/O1024$ à $C = 4$. 35
4.8 Généralisation des résultats à une augmentation de la taille du modèle pour $P128/O1024$ à $C = 4$. 37
4.9 Généralisation des résultats à une autre famille de modèles pour $P128/O1024$ à $C = 4$. 38
4.10 Influence de la quantification sur les performances, les ressources et l'énergie pour Qwen2.5-1.5B-Instruct à $P128/O1024$ et $C = 4$. 39
4.11 Généralisation de l'effet de la charge sur les performances côté client entre l'AGX Orin et l'Orin Nano à $C = 1$. 41
4.12 Généralisation de l'effet de la charge sur les ressources mobilisées et l'énergie entre l'AGX Orin et l'Orin Nano à $C = 1$. 42
4.13 Généralisation de l'effet de la concurrence sur les performances côté client entre l'AGX Orin et l'Orin Nano pour $P128/O1024$. 43
4.14 Généralisation de l'effet de la concurrence sur les ressources mobilisées et l'énergie entre l'AGX Orin et l'Orin Nano pour $P128/O1024$. 44
4.15 Généralisation de l'effet de la quantification entre l'AGX Orin et l'Orin Nano pour Qwen2.5-1.5B-Instruct, $P128/O1024$ et $C = 4$. 45

Liste des tableaux

2.1 Principales approches des systèmes d'inférence présentés. 15
2.2 Dimensions couvertes par les travaux de caractérisation étudiés. 18

3.1 Caractéristiques des plateformes expérimentales. 20
3.2 Configurations de référence de `llama.cpp` et vLLM. 21

4.1 Synthèse des principaux résultats de Q1 à Q5. 46

Acronymes

CPU *Central Processing Unit*. 14, 15, 17, 20, 21, 23, 24, 28, 29, 32, 33, 35, 36, 38–40, 42, 45–49

FCFS *First-Come, First-Served*. 10

FFN *Feed-Forward Network*. 5, 6

GPU *Graphics Processing Unit*. 8, 14–17, 20, 21, 23, 24, 28, 31–33, 35, 38–40, 42, 43, 49

KV *Key-Value*. v, 4, 7–12, 14–17, 21, 28

LLM *Large Language Models*. iii, v, 1–16, 18, 19, 33, 34, 36, 46, 49

MHA *Multi-Head Attention*. 6

SJF *Shortest Job First*. 10

SLM *Small Language Models*. 11

TPOT *Time Per Output Token*. 22, 24, 28, 29, 31–33, 35–40, 42–48

TTFT *Time To First Token*. 16, 22, 24, 27–31, 34, 36–40, 42–44, 46–48

Introduction générale

Contexte

Les modèles de langage de grande taille (LLM) occupent aujourd'hui une place importante dans de nombreuses applications d'intelligence artificielle, telles que les assistants conversationnels, la génération de contenu, la traduction ou encore la génération de code [28]. Leur utilisation repose cependant sur des besoins importants en calcul et en mémoire, ce qui explique que leur déploiement soit encore largement associé à des infrastructures *Cloud* disposant de ressources matérielles conséquentes [28, 7, 5]. Avec l'augmentation du nombre d'utilisateurs de ces services, une même infrastructure est également amenée à traiter plusieurs requêtes simultanément [16, 24]. Le déploiement multi-utilisateur, ou *multi-tenant*, permet alors de mutualiser une même instance du modèle et les ressources matérielles associées entre plusieurs utilisateurs, plutôt que de leur attribuer des ressources séparées [13, 18]. Cette mutualisation vise à améliorer l'utilisation de l'infrastructure, mais elle introduit en contrepartie des enjeux liés à la concurrence entre les requêtes, au partage des ressources et au maintien de bonnes performances pour les utilisateurs [13, 2, 18].

Ces enjeux deviennent d'autant plus importants lorsque l'inférence est déplacée du *Cloud* vers des plateformes *Edge* [28, 7]. L'exécution au plus près des utilisateurs et des données permet notamment de réduire certaines latences de communication, de limiter la dépendance au *Cloud* et de conserver un meilleur contrôle sur les données traitées [7, 1, 10]. Elle impose toutefois de composer avec des capacités de calcul et de mémoire plus limitées que celles disponibles dans des infrastructures serveur [28, 7, 1]. Dans un environnement *Edge* multi-utilisateur, il faut donc parvenir à traiter efficacement plusieurs requêtes tout en maîtrisant l'utilisation de ressources contraintes, faisant ainsi apparaître un compromis direct entre performances et utilisation des ressources [22, 30].

L'étude de ce type de compromis s'inscrit dans les thématiques du Lab-STICC liées à l'optimisation des systèmes matériels et logiciels. Le Lab-STICC, Laboratoire des Sciences et Techniques de l'Information, de la Communication et de la Connaissance, est une unité de recherche pluridisciplinaire du CNRS organisée autour de plusieurs pôles de recherche. Le stage est réalisé à l'ENSTA, sur le campus de Brest, au sein du pôle SHARP (*Software and Hardware, ARchitectures and Processes*), et plus particulièrement dans l'équipe SHAKER, spécialisée dans l'optimisation des systèmes matériels et logiciels.

Dans ce contexte, le comportement du système dépend notamment de la manière dont l'exécution des requêtes et l'utilisation des ressources sont gérées au niveau logiciel [16]. Ce rôle est assuré par les moteurs d'inférence, qui intègrent différents mécanismes tels que le regroupement et l'ordonnancement des requêtes, la gestion de la mémoire ou encore l'optimisation de l'exécution du modèle [16, 13, 2]. Ces moteurs peuvent cependant suivre des choix de conception différents [16]. Parmi les solutions largement utilisées, `vLLM` [13] est conçu avec une forte orientation vers le traitement de requêtes concurrentes et l'obtention

d'un débit élevé, tandis que `llama.cpp` [9] privilégie davantage la légèreté, la portabilité et l'exécution sur une grande diversité de matériels, notamment des plateformes aux ressources contraintes.

Ces deux orientations répondent ainsi à des besoins qui se rencontrent précisément dans un environnement *Edge* multi-utilisateur : maintenir de bonnes performances lorsque plusieurs requêtes sont traitées simultanément tout en limitant l'utilisation des ressources disponibles [22, 30]. Le choix du moteur le plus adapté ne peut donc pas être déterminé uniquement à partir de leurs caractéristiques générales et peut varier selon les conditions d'utilisation [15]. Il devient alors nécessaire de caractériser expérimentalement leur comportement afin de comprendre les compromis qu'ils présentent et d'identifier les mécanismes susceptibles d'expliquer les différences observées [15, 1].

Les travaux existants abordent cependant encore largement ces dimensions de manière séparée [5, 24, 7, 1]. Les études consacrées à l'inférence multi-utilisateur évaluent principalement des environnements serveur disposant de ressources importantes [18, 5, 24], tandis que les travaux menés sur des plateformes *Edge* se concentrent davantage sur l'influence de la charge, du modèle, de la quantification ou de la plateforme [7, 1, 10, 11]. Les études combinant à la fois contexte *Edge*, concurrence multi-utilisateur, performances côté utilisateur et ressources système restent ainsi limitées [22, 30]. C'est précisément à cette intersection que se situe ce travail et que s'inscrit la problématique formulée ci-après.

Problématique

L'étude porte ainsi sur la manière dont les compromis entre performances perçues par les utilisateurs et utilisation des ressources évoluent selon les caractéristiques de la charge de travail et les conditions de partage de la plateforme.

Dans ce contexte, la problématique étudiée dans ce stage est la suivante :

> *Dans un contexte Edge multi-utilisateur, comment les différents moteurs d'inférence se comportent-ils en termes de performances perçues par l'utilisateur et d'utilisation des ressources sous différents types de charges de travail ?*

Objectifs du stage

Afin de répondre à cette problématique, le stage se concentre sur la caractérisation expérimentale de `llama.cpp` et `vLLM`. L'objectif est de comprendre leurs comportements respectifs, d'identifier les facteurs qui influencent leurs performances et leur utilisation des ressources, et de fournir une base expérimentale pouvant guider de futures stratégies d'optimisation.

Les principaux objectifs sont les suivants :

1. étudier les travaux existants sur l'inférence multi-utilisateur des LLM, sa caractérisation et les techniques d'optimisation associées ;
2. mettre en place une plateforme expérimentale reproductible permettant d'exécuter et de mesurer `llama.cpp` et `vLLM` sur des plateformes *Edge* ;
3. caractériser l'influence des principales propriétés de la charge de travail, notamment les longueurs d'entrée et de sortie ainsi que le niveau de concurrence, sur les performances perçues par l'utilisateur et l'utilisation des ressources matérielles ;

4. vérifier dans quelle mesure les tendances observées se maintiennent lorsque d'autres paramètres du déploiement évoluent, notamment la famille et la taille du modèle, sa quantification, la plateforme matérielle et le partage des ressources avec une autre application ;
5. identifier les principaux compromis et limites des deux moteurs afin de dégager des éléments pouvant servir de base à de futures stratégies d'optimisation et d'ordonnancement.

Organisation du rapport

Le rapport est organisé comme suit :

— **Chapitre 1 : Inférence des LLM sur *Edge* :** présentation des principes de l'inférence des LLM, puis des mécanismes liés à l'inférence multi-utilisateur, notamment l'ordonnancement des requêtes et la gestion de la mémoire. Les principales techniques d'adaptation des modèles aux ressources limitées des plateformes *Edge* sont également présentées.

— **Chapitre 2 : État de l'art :** présentation des systèmes et techniques d'optimisation de l'inférence des LLM, puis analyse des travaux de caractérisation existants en contexte multi-utilisateur et *Edge* afin de positionner ce travail.

— **Chapitre 3 : Méthodologie expérimentale :** présentation des questions de recherche, des plateformes et configurations utilisées, des métriques et outils de mesure ainsi que du protocole expérimental commun aux différentes campagnes.

— **Chapitre 4 : Campagnes expérimentales, résultats et analyse :** présentation des expérimentations menées pour étudier successivement l'influence de la charge de travail, de la concurrence et du partage des ressources, puis la généralisation des observations à différents modèles, quantifications et à une seconde plateforme *Edge*.

— **Conclusion générale :** synthèse des principaux enseignements de l'étude, des difficultés rencontrées et de ses limites, suivie des perspectives de prolongement du travail.

Chapitre 1

Inférence des LLM sur *Edge*

1.1 Introduction

L'inférence des LLM correspond à la phase d'utilisation d'un modèle de langage après son entraînement, durant laquelle il traite une entrée et génère progressivement une réponse [21]. Dans un contexte *Edge* multi-utilisateur, plusieurs requêtes peuvent être traitées simultanément sur une même plateforme et partager des ressources de calcul et de mémoire limitées, ce qui rend essentielle la compréhension du fonctionnement de l'inférence pour analyser les performances du système [28, 7].

Ce chapitre présente les notions nécessaires à la compréhension du travail expérimental réalisé dans ce mémoire. Il décrit d'abord l'architecture et le processus d'inférence des LLM, en distinguant notamment les phases de *prefill* et de *decode* ainsi que le rôle du cache KV, puis présente les principaux mécanismes permettant de traiter plusieurs requêtes simultanément, en particulier l'ordonnancement et la gestion de la mémoire. Enfin, les principales techniques utilisées pour adapter les LLM aux ressources limitées des plateformes *Edge* sont introduites.

1.2 Principes de l'inférence des LLM

L'inférence d'un LLM repose à la fois sur l'architecture du modèle et sur le processus utilisé pour générer une réponse à partir d'une entrée. Cette section présente d'abord les éléments de l'architecture nécessaires à la compréhension de l'inférence, puis décrit le processus de génération et ses principales étapes.

1.2.1 Architecture des LLM

Un modèle de langage apprend les régularités d'un langage afin d'estimer la probabilité de la suite d'un texte à partir du contexte qui le précède. Les modèles de langage de grande taille, ou *Large Language Models* (LLM), reposent sur ce principe en utilisant des réseaux de neurones comportant un grand nombre de paramètres, c'est-à-dire de valeurs apprises lors de l'entraînement, à partir de volumes importants de données textuelles [21].

La majorité des LLM génératifs actuels reposent sur l'architecture Transformer, introduite par Vaswani et al. [23]. Cette architecture organise le réseau sous la forme d'une succession de blocs de traitement dont le fonctionnement repose principalement sur un mécanisme d'attention permettant de mettre en relation les différents éléments d'une séquence et de déterminer leur importance en fonction du contexte [23].

Selon l'organisation des blocs Transformer, trois grandes familles d'architectures peuvent être distinguées : *encoder-only*, *encoder-decoder* et *decoder-only* [21]. Les architectures *encoder-only* sont principalement utilisées pour analyser et représenter une séquence d'entrée, notamment dans des tâches de classification ou d'extraction d'information. Les architectures *encoder-decoder* associent un encodeur chargé de construire une représentation de l'entrée à un décodeur qui produit la séquence de sortie. Enfin, les architectures *decoder-only* reposent uniquement sur une succession de blocs de type décodeur et génèrent le texte de manière autorégressive, c'est-à-dire un token après l'autre en utilisant les tokens précédents comme contexte [21].

Dans ce travail, nous nous intéressons principalement aux architectures de type *decoder-only*, largement utilisées par les LLM génératifs actuels et auxquelles appartiennent les modèles étudiés dans ce mémoire [21]. Leur fonctionnement nécessite tout d'abord de transformer le texte en une représentation pouvant être traitée par le réseau.

Avant d'être traité par le modèle, le texte est découpé par un *tokenizer* en unités appelées *tokens*. Selon la méthode de tokenisation utilisée, un token peut correspondre à un mot, une partie de mot ou encore un caractère. Chaque token est ensuite converti en une représentation numérique appelée *embedding*. Une information de position est également intégrée au traitement afin de permettre au modèle de tenir compte de l'ordre des tokens dans la séquence [23, 21].

L'architecture générale d'un Transformer de type *decoder-only* est illustrée dans la Figure 1.1.

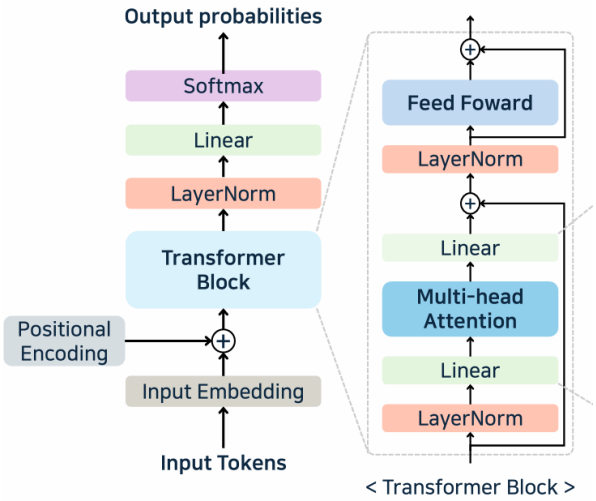


FIGURE 1.1 – Architecture simplifiée d'un Transformer de type *decoder-only*, adaptée de [21].

Comme illustré dans la Figure 1.1, les représentations des tokens traversent une succession de blocs Transformer. Chaque bloc repose principalement sur trois éléments : le mécanisme d'attention, le réseau *Feed-Forward Network* (FFN), ainsi que les opérations de normalisation et les connexions résiduelles [23].

1. **Mécanisme d'attention.**
   Le mécanisme d'attention permet au modèle de mettre en relation les différents tokens de la séquence et de déterminer quelles informations du contexte sont les plus pertinentes pour traiter un token donné. Pour cela, les représentations en entrée sont

projetées en trois ensembles de vecteurs : les requêtes (*Queries*, Q), les clés (*Keys*, K) et les valeurs (*Values*, V) [23].

Intuitivement, une requête représente l'information recherchée par un token, les clés permettent d'évaluer la pertinence des autres tokens par rapport à cette requête, tandis que les valeurs contiennent les informations qui seront effectivement combinées. Dans une architecture *decoder-only*, l'attention peut être exprimée sous la forme [23] :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V, \tag{1.1}$$

où d_k représente la dimension des vecteurs de clés et M le masque causal. Ce dernier empêche un token de prendre en compte les tokens situés après lui dans la séquence, de manière à préserver le caractère autorégressif de la génération. Le produit QK^T fournit des scores représentant la relation entre les requêtes et les clés. Après application du masque causal, la fonction *softmax* transforme ces scores en poids normalisés, qui sont ensuite utilisés pour pondérer les vecteurs de valeurs V et construire une représentation tenant compte du contexte disponible [23].

Dans l'architecture Transformer originale, plusieurs calculs d'attention, appelés *têtes d'attention*, sont réalisés en parallèle selon le mécanisme *Multi-Head Attention* (MHA). Chaque tête peut ainsi capturer des relations différentes entre les tokens avant que leurs résultats ne soient combinés [23].

2. **Réseau *Feed-Forward Network*.**
   La sortie du mécanisme d'attention est ensuite traitée par un réseau *Feed-Forward Network* (*Feed-Forward Network* (FFN)). Dans sa forme générale, celui-ci est appliqué indépendamment à la représentation de chaque token et repose sur plusieurs transformations linéaires associées à une fonction d'activation non linéaire. Une première transformation projette généralement la représentation dans un espace de dimension plus élevée avant qu'elle ne soit ramenée à sa dimension initiale. Le FFN permet ainsi de transformer et d'enrichir les représentations produites par le mécanisme d'attention [23].
3. **Normalisation et connexions résiduelles.**
   Les blocs Transformer utilisent également des opérations de normalisation afin de contrôler l'échelle des représentations et de stabiliser leur propagation à travers les différentes couches du réseau. Des connexions résiduelles permettent parallèlement de réinjecter l'entrée d'un sous-bloc dans sa sortie et de faciliter la propagation de l'information à travers les nombreuses couches du modèle [23].

Après avoir traversé l'ensemble des blocs Transformer, la représentation obtenue est projetée sur le vocabulaire du modèle, c'est-à-dire l'ensemble des tokens qu'il peut représenter, afin de calculer une distribution de probabilités sur les différents tokens possibles. Cette prédiction constitue l'opération élémentaire à partir de laquelle une réponse complète est générée, selon le processus présenté dans la sous-section suivante [21].

1.2.2 Processus d'inférence

L'inférence d'un LLM génératif est un processus itératif : le modèle génère successivement les tokens composant la réponse [21]. Pour une séquence de tokens $x_1, \ldots, x_n$, ce fonctionnement autorégressif peut être exprimé par la factorisation suivante :

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}), \tag{1.2}$$

où $P(x_i \mid x_1, \ldots, x_{i-1})$ représente la probabilité du token x_i connaissant les tokens qui le précèdent.

Lors de l'inférence, le texte fourni par l'utilisateur, appelé *prompt*, constitue le contexte initial. À partir de ce contexte, le modèle calcule une distribution de probabilités sur son vocabulaire afin de sélectionner le prochain token. Celui-ci est ensuite ajouté au contexte et utilisé avec les tokens précédents pour générer le suivant. Ce processus est répété jusqu'à la génération d'un token de fin de séquence ou jusqu'à ce qu'une limite de génération soit atteinte. Cette dépendance entre les tokens impose un caractère séquentiel à la génération, puisqu'un nouveau token ne peut être produit qu'une fois le précédent disponible [21].

Bien que la génération soit autorégressive, le traitement du contexte initial et la génération des tokens suivants ne présentent pas les mêmes caractéristiques d'exécution. Comme illustré dans la Figure 1.2, le processus d'inférence se décompose ainsi principalement en deux phases : la phase de *prefill*, consacrée au traitement du contexte d'entrée et à la génération du premier token, puis la phase de *decode*, durant laquelle les tokens suivants sont générés successivement [21]. Ces deux phases présentent des caractéristiques de calcul et de mémoire différentes et s'appuient sur l'utilisation du cache KV [21, 13].

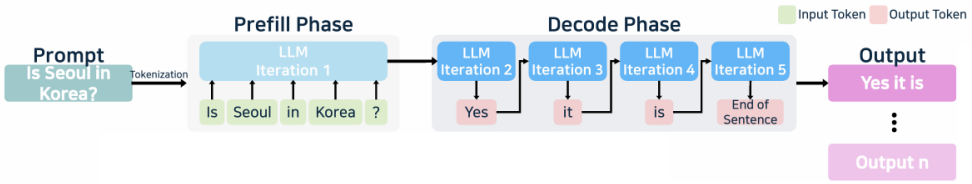


FIGURE 1.2 – Processus d'inférence autorégressive d'un LLM, adapté de [21].

Phase de *prefill*

La phase de *prefill* correspond au traitement du contexte initial fourni par l'utilisateur. L'ensemble des tokens du *prompt* est traité par les blocs Transformer afin de construire leurs représentations et de produire les informations nécessaires à la génération du premier token [21].

Les tokens du *prompt* étant disponibles dès le début de l'inférence, les opérations qui leur sont associées peuvent être réalisées en parallèle. Les principales opérations matricielles sont ainsi appliquées à plusieurs tokens simultanément, ce qui favorise une bonne exploitation des unités de calcul disponibles. La quantité de calcul réalisée pendant le *prefill* dépend toutefois fortement de la longueur du contexte d'entrée : lorsque celle-ci augmente, le volume de calcul et la quantité de données manipulées augmentent également [21].

Phase de *decode*

Après la génération du premier token, l'inférence se poursuit par la phase de *decode*. À chaque itération, le dernier token généré est traité par les différentes couches du modèle

afin de produire un unique nouveau token, en tenant compte du contexte accumulé jusque-là [21].

Contrairement au *prefill*, le parallélisme disponible au sein d'une requête est plus faible, puisque chaque nouveau token dépend du précédent. Les paramètres du modèle et les informations associées au contexte doivent par ailleurs être consultés à chaque étape de génération. Le *decode* est donc généralement davantage contraint par les accès mémoire, notamment parce que ces données doivent être relues à chaque itération [21]. Une sortie plus longue implique ainsi davantage d'itérations de *decode* et donc davantage d'accès répétés aux données nécessaires à la génération [21].

Cache KV

Lors de la génération autorégressive, recalculer à chaque itération les clés et les valeurs correspondant à l'ensemble des tokens précédents entraînerait de nombreux calculs redondants. Pour éviter ces recalculs, les systèmes d'inférence utilisent un *Key-Value cache*, ou cache KV. Pendant le *prefill*, les clés K et les valeurs V produites pour les tokens du *prompt* sont conservées en mémoire. À chaque itération de *decode*, seules celles correspondant au nouveau token sont calculées puis ajoutées au cache, tandis que les informations relatives aux tokens précédents peuvent être directement réutilisées lors du calcul de l'attention [13, 21].

Cette réutilisation réduit le volume de calcul nécessaire pendant le *decode*, au prix d'une occupation mémoire supplémentaire. Le cache doit également être consulté à chaque étape de génération. Sa taille augmente avec la longueur du contexte et dépend également de l'architecture ainsi que du format numérique utilisé pour stocker les clés et les valeurs [21, 13]. Dans un environnement multi-utilisateur, plusieurs caches doivent être maintenus simultanément, ce qui augmente les besoins en mémoire et peut limiter le nombre de requêtes actives [13].

1.3 Inférence multi-utilisateur

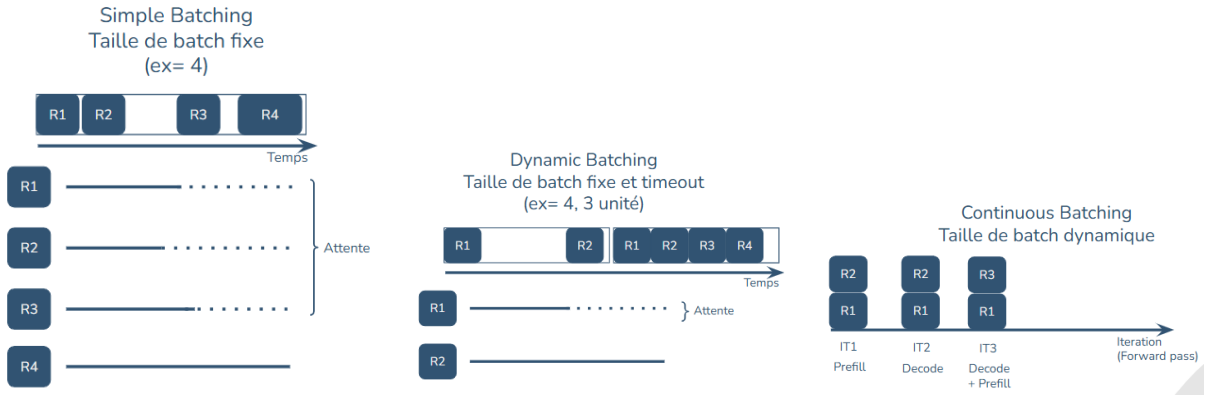
En plus des contraintes liées à la mémoire et au cache KV, l'inférence des LLM pose également un problème d'utilisation des ressources de calcul. Le faible parallélisme disponible lors de la génération peut conduire à une sous-utilisation du GPU, ce qui motive, dans un contexte multi-utilisateur, le traitement simultané de plusieurs requêtes afin de mieux exploiter les ressources matérielles [25, 13]. Cette exécution conjointe est cependant plus complexe qu'une succession de requêtes indépendantes puisque les requêtes peuvent arriver à des instants différents, progresser à des rythmes différents et nécessiter des quantités de mémoire variables au cours de leur génération. Il faut donc organiser leur exécution tout en tenant compte des ressources disponibles [25, 17]. Dans ce travail, nous nous intéressons principalement à deux aspects de cette gestion : l'ordonnancement des requêtes et la gestion de la mémoire.

1.3.1 Ordonnancement des requêtes

Lorsque plusieurs requêtes sont présentes simultanément, leur nombre définit le niveau de concurrence, mais elles ne sont pas nécessairement exécutées toutes en même temps. L'ordonnancement détermine lesquelles exécuter et à quel moment afin d'exploiter davantage le parallélisme disponible, ce qui conduit à regrouper certaines d'entre elles au sein

d'un même groupe, appelé *batch*. La manière dont ce groupe est constitué et évolue au cours de l'exécution dépend alors de la stratégie de *batching* utilisée [17]. Trois principales stratégies sont distinguées ici et illustrées dans la Figure 1.3 :

1. ***Batching* statique :** La forme la plus simple est le *batching* statique, illustré dans la Figure 1.3a. Il utilise un nombre fixe de requêtes par groupe : l'exécution commence lorsque ce nombre est atteint, puis le groupe reste inchangé jusqu'à la terminaison de toutes les requêtes. Cette approche peut introduire une attente avant le début de l'inférence puisqu'il faut d'abord constituer un groupe complet et une attente à la fin de l'inférence lorsque les longueurs de sortie diffèrent : une requête courte peut terminer sa génération bien avant les autres, tandis que le groupe reste occupé jusqu'à la terminaison de la requête la plus longue [17].
2. ***Batching* dynamique :** Le *batching* dynamique, présenté dans la Figure 1.3b, cherche notamment à réduire l'attente nécessaire pour constituer un groupe complet. Au lieu d'attendre systématiquement un nombre fixe de requêtes, celles-ci sont regroupées pendant une courte période et l'exécution commence lorsque la taille maximale du groupe est atteinte ou lorsque ce délai expire. La taille du *batch* peut ainsi varier selon les requêtes disponibles. Une fois l'exécution commencée, sa composition reste toutefois fixe, de sorte que les requêtes courtes restent contraintes par celles dont la génération est plus longue [17].
3. ***Batching* continu :** Le *batching* continu, ou *continuous batching*, illustré dans la Figure 1.3c, permet au contraire de faire évoluer la composition du groupe pendant la génération. Il repose sur un ordonnancement à l'échelle des itérations, ou *iteration-level scheduling* : à chaque itération de *decode*, les requêtes terminées peuvent quitter le groupe et être remplacées par de nouvelles requêtes en attente, sans attendre la terminaison des autres séquences [25]. Le *batching* continu est aujourd'hui largement adopté dans les systèmes d'inférence des LLM et constitue un mécanisme courant pour améliorer l'utilisation des ressources lors du traitement de requêtes concurrentes [17].



(a) *Batching* statique (b) *Batching* dynamique (c) *Batching* continu

FIGURE 1.3 – Comparaison des stratégies de *batching* statique, dynamique et continu.

Le *batching* continu ne permet toutefois pas de maintenir un nombre illimité de requêtes actives, puisque cette capacité dépend notamment de la mémoire disponible pour les séquences et leurs caches KV ainsi que de la quantité de travail pouvant être traitée

à chaque itération [13, 17]. Lorsque cette capacité est atteinte, les requêtes supplémentaires restent en attente. À mesure que des ressources se libèrent, le système doit alors déterminer lesquelles admettre dans le *batch* actif, selon une politique d'ordonnancement pouvant reposer sur différents critères [17, 8] :

— **Ordre d'arrivée : *First-Come, First-Served* (FCFS).**
La politique *First-Come, First-Served* (FCFS) sélectionne les requêtes selon leur ordre d'arrivée. Elle est simple à mettre en œuvre puisqu'elle ne nécessite aucune information supplémentaire sur les requêtes. Cependant, une requête longue arrivée en premier peut retarder les requêtes plus courtes qui la suivent, phénomène appelé *Head-of-Line blocking* [8].

— **Durée de traitement : *Shortest Job First* (SJF).**
Une politique de type *Shortest Job First* (SJF) cherche au contraire à favoriser les requêtes dont le traitement est le plus court afin de réduire leur temps d'attente. Son application aux LLM est toutefois difficile, car la longueur de la réponse n'est généralement pas connue avant sa génération. Certains travaux cherchent donc à estimer cette longueur ou son ordre relatif afin d'approcher ce type d'ordonnancement [8].

— **Priorité des requêtes.**
Une priorité peut également être associée à chaque requête afin de traiter en premier celles considérées comme les plus importantes. Cette priorité peut par exemple refléter différents niveaux de service ou différentes contraintes de latence. Lorsque plusieurs requêtes possèdent la même priorité, leur ordre d'arrivée peut être utilisé pour les départager [17].

L'ordonnancement permet ainsi d'augmenter le travail exécuté en parallèle, mais le nombre de requêtes pouvant être maintenues simultanément reste notamment contraint par la mémoire nécessaire à leur exécution, ce qui conduit au second aspect du traitement multi-utilisateur : la gestion de la mémoire [13].

1.3.2 Gestion de la mémoire

Comme présenté précédemment, le cache KV d'une requête augmente progressivement avec le nombre de tokens de son contexte. Après le *prefill*, il contient les informations associées aux tokens du *prompt*, puis il s'agrandit d'un token à chaque itération de *decode*. La longueur finale de la réponse n'étant pas connue à l'avance, la taille finale du cache KV d'une requête ne peut donc pas être déterminée précisément au début de son exécution [13].

Dans un contexte multi-utilisateur, plusieurs caches KV évoluent ainsi simultanément et indépendamment. Une approche classique consiste à réserver à chaque requête une zone mémoire contiguë suffisamment grande pour contenir son cache jusqu'à une longueur maximale. Cette allocation simplifie la gestion de la mémoire, mais peut entraîner deux formes de fragmentation [13], illustrées dans la Figure 1.4 :

— **Fragmentation interne :** une partie de la zone réservée à une requête reste inutilisée lorsque sa séquence se termine avant d'atteindre la taille maximale prévue.

— **Fragmentation externe :** lorsque des requêtes de tailles différentes terminent leur exécution, les zones libérées peuvent être dispersées en plusieurs espaces non contigus. La quantité totale de mémoire libre peut alors être suffisante pour une nouvelle requête sans qu'une zone contiguë suffisamment grande soit disponible.

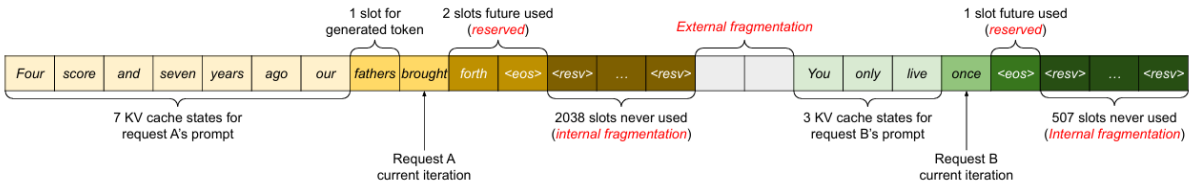


FIGURE 1.4 – Fragmentation interne et externe lors d'une allocation contiguë des caches KV de plusieurs requêtes, adaptée de [13].

Pour limiter ces problèmes, PagedAttention propose une gestion du cache KV inspirée de la mémoire virtuelle [13]. Au lieu de réserver une grande zone contiguë pour chaque requête, le cache est découpé en blocs de taille fixe. Du point de vue de la requête, ces blocs forment une suite de *blocs logiques*, tandis qu'ils peuvent être placés dans des *blocs physiques* non contigus en mémoire [13].

Une table de blocs, ou *block table*, maintient la correspondance entre les blocs logiques d'une requête et leur emplacement physique. Les blocs sont alloués progressivement à mesure que la séquence s'allonge et peuvent être libérés puis réutilisés lorsqu'une requête termine. Cette organisation évite ainsi de réserver dès le départ une grande zone contiguë pour chaque séquence et réduit fortement la fragmentation de la mémoire [13].

Cette organisation par blocs permet de mieux utiliser la mémoire disponible et de maintenir davantage de requêtes actives simultanément [13]. L'ordonnancement et la gestion de la mémoire présentés dans cette section ne constituent cependant pas des mécanismes isolés, puisqu'ils sont intégrés dans des logiciels dédiés à l'exécution des LLM, appelés moteurs d'inférence, qui seront présentés dans l'état de l'art, Section 2.2.

Une grande partie de ces approches a été développée pour des environnements serveur disposant de ressources importantes, tandis que leur utilisation sur des plateformes *Edge* impose des contraintes plus fortes en calcul, en mémoire et en énergie, ce qui nécessite des adaptations spécifiques présentées dans la section suivante [28, 7].

1.4 Adaptation des modèles aux plateformes *Edge*

Les ressources de calcul et de mémoire limitées des plateformes *Edge* conduisent à réduire le coût d'exécution des LLM avant leur déploiement. Plusieurs approches peuvent être utilisées à cette fin, notamment l'utilisation de modèles plus compacts, la quantification, le *pruning* et la distillation [29].

— **Modèles compacts.** Une première approche consiste à utiliser directement des modèles comportant moins de paramètres. Les *Small Language Models* (SLM) cherchent ainsi à conserver des capacités de compréhension et de génération tout en réduisant les besoins en calcul et en mémoire, ce qui facilite leur exécution sur des plateformes aux ressources limitées [20].

— **Quantification.** La quantification consiste à représenter les paramètres du modèle avec un nombre réduit de bits, par exemple en passant d'une représentation FP16 à des formats 8 bits ou 4 bits. Cette réduction diminue directement la mémoire nécessaire au stockage du modèle et le volume de données manipulé pendant l'inférence, tandis que les gains de performance dépendent de la capacité du moteur et du matériel à exploiter efficacement le format considéré. Elle introduit également une

approximation numérique dont l'impact dépend du modèle et du format utilisé [29, 21].

— ***Pruning.*** Le *pruning*, ou élagage, consiste à supprimer des paramètres ou des structures considérés comme peu importants afin de réduire la taille du modèle et la quantité de calcul nécessaire à son exécution. Il peut être appliqué à des poids individuels ou à des structures complètes du réseau, comme certaines têtes d'attention ou couches. Les gains obtenus à l'exécution dépendent toutefois de la capacité du matériel et du moteur à exploiter la structure obtenue [29].

— **Distillation.** La distillation consiste à entraîner un modèle plus petit, appelé *student*, à reproduire le comportement d'un modèle plus important, appelé *teacher*. Elle permet ainsi d'obtenir un modèle nécessitant moins de mémoire et de calcul tout en cherchant à conserver une partie des capacités du modèle initial [29].

1.5 Conclusion

Ce chapitre a présenté les principaux éléments nécessaires pour comprendre l'inférence des LLM dans un contexte *Edge* multi-utilisateur. Le fonctionnement des phases de *prefill* et de *decode* ainsi que l'utilisation du cache KV permettent de comprendre les contraintes qui apparaissent lorsque plusieurs requêtes sont traitées simultanément, notamment en matière d'ordonnancement et de gestion de la mémoire. Sur *Edge*, ces contraintes sont renforcées par des ressources de calcul et de mémoire plus limitées, ce qui conduit également à adapter les modèles afin de réduire leur coût d'exécution.

Ces éléments permettent d'aborder les solutions proposées pour améliorer l'inférence multi-utilisateur et son exécution sur *Edge*, ainsi que les travaux qui cherchent à caractériser expérimentalement le comportement de ces systèmes, présentés dans le chapitre suivant.

Chapitre 2

État de l'art

2.1 Introduction

Différentes solutions ont été proposées pour améliorer l'exécution des LLM dans un contexte multi-utilisateur et sur des plateformes *Edge* [16, 28]. En parallèle, des travaux de caractérisation ont étudié l'influence de la charge de travail, du moteur ou de la plateforme sur les performances et l'utilisation des ressources [15, 7].

Ce chapitre présente d'abord les principales solutions proposées dans ces deux contextes, puis les travaux de caractérisation existants afin de positionner l'étude menée dans ce mémoire.

2.2 Systèmes et optimisations pour l'inférence des LLM

Les systèmes d'inférence des LLM ont été développés pour répondre à des contraintes différentes selon leur contexte d'utilisation [16, 21]. Dans un environnement multi-utilisateur, les travaux cherchent notamment à organiser efficacement l'exécution de plusieurs requêtes et leur partage des ressources, tandis que sur les plateformes *Edge*, les capacités matérielles plus limitées conduisent à privilégier des solutions adaptées à ces contraintes [16, 28]. Cette section présente les principales approches proposées dans ces deux contextes afin de mettre en évidence les mécanismes qu'elles utilisent et les contraintes auxquelles elles cherchent à répondre.

2.2.1 Solutions adaptées au contexte multi-utilisateur

Dans un contexte multi-utilisateur, l'efficacité de l'inférence dépend de la capacité du système à faire progresser plusieurs requêtes tout en utilisant efficacement les ressources disponibles. Les travaux existants agissent principalement sur l'ordonnancement des requêtes et la gestion de la mémoire [16].

Orca [26] adapte explicitement l'ordonnancement au caractère itératif de la génération autorégressive. Il introduit l'*iteration-level scheduling*, présenté dans la Section 1.3.1, afin de permettre à la composition du *batch* d'évoluer au cours de la génération plutôt que de rester fixe jusqu'à la terminaison de toutes les requêtes. Cette granularité d'ordonnancement permet notamment de mieux traiter des requêtes dont les longueurs diffèrent et constitue l'un des principes repris par les systèmes utilisant aujourd'hui le *batching* continu.

Cette évolution de l'ordonnancement ne supprime toutefois pas les interactions entre les phases de *prefill* et de *decode* lorsque plusieurs requêtes progressent simultanément. Un *prefill* long peut mobiliser les ressources pendant une durée importante et retarder les itérations de *decode* des requêtes déjà actives, ce qui conduit Sarathi-Serve [2] à découper le *prefill* en plusieurs fragments plus petits, appelés *chunks*, afin de les intégrer progressivement aux itérations de génération. Cette organisation permet de limiter les interruptions du *decode* tout en conservant suffisamment de travail parallèle pour maintenir un débit élevé, de manière à améliorer le compromis entre débit et latence sous charge concurrente.

L'augmentation du nombre de requêtes actives accentue également les besoins en mémoire, notamment en raison de l'accumulation de leurs caches KV. vLLM [13] traite ce problème à travers PagedAttention, dont le principe a été présenté dans la Section 1.3.2, afin de réduire la mémoire inutilisée et de permettre à davantage de séquences d'être maintenues simultanément. Cette gestion mémoire est associée à un ordonnancement permettant l'admission continue de nouvelles requêtes, ce qui oriente vLLM vers les charges comportant un nombre important de requêtes concurrentes.

Lorsque plusieurs requêtes partagent une partie de leur contexte, certains calculs peuvent également être évités en réutilisant les informations déjà produites. SGLang [27] exploite cette possibilité avec RadixAttention, qui conserve les parties du cache KV associées aux préfixes déjà traités afin de les réutiliser lorsqu'elles apparaissent dans de nouvelles requêtes. Cette approche réduit ainsi les calculs redondants lorsque plusieurs requêtes possèdent un contexte commun.

Ces travaux montrent ainsi que le traitement de requêtes concurrentes peut être amélioré à plusieurs niveaux complémentaires : Orca [26] et Sarathi-Serve [2] agissent principalement sur l'ordonnancement des requêtes, tandis que vLLM [13] et SGLang [27] cherchent également à mieux exploiter la mémoire et les informations déjà calculées. Ces solutions ont toutefois été principalement développées pour des environnements serveur, alors que leur utilisation sur des plateformes *Edge* impose des contraintes de calcul et de mémoire plus fortes et conduit à rechercher des systèmes adaptés à ces ressources [28].

2.2.2 Solutions adaptées aux plateformes *Edge*

Au-delà des techniques appliquées directement au modèle présentées dans la Section 1.4, son exécution sur *Edge* dépend également de la capacité du système d'inférence à s'adapter au matériel disponible [28]. Certaines solutions privilégient ainsi une implémentation légère et portable, tandis que d'autres adaptent l'exécution à la plateforme ciblée.

`llama.cpp` [9, 21] s'inscrit dans cette logique à travers une implémentation en C/C++ comportant peu de dépendances et compatible avec différentes architectures CPU et GPU, notamment x86 et ARM. Il s'appuie notamment sur le format GGUF, qui regroupe les paramètres du modèle ainsi que les informations nécessaires à son exécution et permet de distribuer des modèles utilisant différents niveaux de quantification. Associée à la prise en charge d'une grande diversité de matériels, cette organisation facilite l'exécution locale de modèles dont la taille est adaptée aux ressources disponibles. En s'appuyant notamment sur `llama.cpp`, Ollama [21] facilite à son tour le déploiement et l'exécution locale des LLM en simplifiant le téléchargement, la configuration et l'utilisation des modèles.

MLC LLM [21] poursuit également un objectif de portabilité, mais adopte une approche différente en adaptant l'exécution du modèle à la plateforme ciblée par compila-

tion. Les opérations peuvent ainsi être transformées en fonction du matériel disponible, ce qui permet au système de couvrir différents environnements, notamment les *Graphics Processing Unit* (GPU), les ordinateurs personnels et les appareils mobiles.

Dans un contexte *Edge* multi-utilisateur, certains travaux agissent directement sur l'ordonnancement des requêtes. SLICE [30] propose ainsi de les différencier selon leur niveau de priorité afin d'adapter l'utilisation des ressources à leurs différentes contraintes. Son objectif reste ainsi centré sur l'amélioration de l'ordonnancement lorsque les requêtes présentent des priorités différentes, ce qui correspond à un cadre plus spécifique que celui de requêtes comparables partageant les mêmes ressources.

2.2.3 Synthèse

Les solutions présentées répondent à des contraintes différentes selon leur contexte d'utilisation : dans les environnements multi-utilisateur, les travaux cherchent principalement à améliorer l'exécution simultanée de plusieurs requêtes à travers l'ordonnancement, la gestion de la mémoire ou la réutilisation des calculs déjà réalisés. Sur *Edge*, les solutions cherchent plutôt à adapter l'exécution aux ressources disponibles à travers des implémentations légères, la portabilité ou l'adaptation au matériel. Certains travaux, comme SLICE [30], commencent également à considérer directement l'ordonnancement de plusieurs requêtes sur des plateformes *Edge*.

Le Tableau 2.1 synthétise les principales propositions des systèmes présentés.

TABLE 2.1 – Principales approches des systèmes d'inférence présentés.

Contexte	**Famille**	**Travaux et mécanismes proposés**
Multi-utilisateur	Ordonnancement	**Orca** [26] : ordonnancement à chaque itération. **Sarathi-Serve** [2] : découpage du *prefill* en fragments intégrés progressivement au *decode*.
	Gestion mémoire et cache	**vLLM** [13] : PagedAttention et admission continue des requêtes. **SGLang** [27] : RadixAttention pour réutiliser les préfixes et leurs caches KV.
Edge	Exécution locale	`llama.cpp` [9, 21] : implémentation légère en C/C++, format GGUF, quantification et prise en charge de différentes architectures *Central Processing Unit* (CPU) et GPU.
	Adaptation au matériel	**MLC LLM** [21] : compilation et adaptation de l'exécution à la plateforme ciblée.
Edge multi-utilisateur	Ordonnancement	**SLICE** [30] : ordonnancement des requêtes selon leur niveau de priorité.

2.3 Caractérisation des systèmes d'inférence

Les solutions présentées dans la section précédente reposent sur des choix différents pour organiser l'exécution des LLM, mais leur conception ne suffit pas à déterminer leur

comportement dans des conditions réelles d'utilisation. Des travaux ont donc adopté une démarche expérimentale afin d'étudier l'influence de la charge de travail, de la plateforme ou du système d'inférence sur les performances obtenues et, dans certains cas, sur l'utilisation des ressources [15, 7]. Dans le cadre de notre problématique, ces travaux peuvent principalement être distingués selon deux contextes : la caractérisation en environnement multi-utilisateur et la caractérisation sur des plateformes *Edge*.

2.3.1 Caractérisation en contexte multi-utilisateur

Dans les environnements serveur, plusieurs travaux ont étudié le comportement des systèmes d'inférence lorsque plusieurs requêtes sont traitées simultanément. Ay et al. [4] comparent ainsi Ollama et vLLM sur un GPU NVIDIA H100 en faisant progressivement augmenter le nombre de requêtes concurrentes. Selon les scénarios considérés, vLLM atteint un débit de 20 à 29 fois supérieur et une latence P95 de 8 à 19 fois inférieure à celle d'Ollama sous forte concurrence. Les auteurs observent également que vLLM reste stable jusqu'à 100 utilisateurs concurrents dans leur campagne, tandis qu'Ollama rencontre un premier goulot d'étranglement autour de dix utilisateurs. Ces résultats montrent que les choix réalisés au niveau du système d'inférence peuvent modifier sa capacité à traiter une charge multi-utilisateur.

Bomhold et al. [5] caractérisent également vLLM sous concurrence sur plusieurs GPU disponibles dans le *cloud*. Sur un NVIDIA L40S, leur charge comportant 50 requêtes concurrentes atteint environ 2687 tokens/s, contre 104 tokens/s en exécution séquentielle. Leur étude montre également que les caractéristiques des requêtes restent déterminantes : l'augmentation de la longueur de sortie agit principalement sur la latence, tandis que l'allongement du *prompt* affecte davantage le temps jusqu'au premier token (*Time To First Token*, TTFT).

Ye et al. [24] montrent enfin que, sous concurrence, les performances dépendent conjointement du *batching*, des caractéristiques des requêtes et de l'utilisation du cache KV.

Ces travaux établissent ainsi que la concurrence, la charge des requêtes et les mécanismes utilisés pour organiser leur exécution influencent directement les performances d'un service LLM multi-utilisateur. Les évaluations présentées sont cependant réalisées sur des GPU de serveurs ou dans des environnements *cloud*.

2.3.2 Caractérisation sur les plateformes *Edge*

D'autres travaux se sont intéressés directement au comportement de l'inférence sur des plateformes *Edge*, où les ressources disponibles sont plus limitées.

Dhar et al. [7] mettent en évidence la dépendance au matériel en exécutant avec `llama.cpp` un modèle LLaMA-2 7B quantifié en INT4 sur plusieurs dispositifs. Le débit passe de 0,11 token/s sur un Raspberry Pi 4 disposant de 8 Go de mémoire à 4,49 tokens/s sur une NVIDIA Jetson AGX Orin. Leur analyse montre également que la ressource limitante peut changer selon la plateforme, notamment lorsque la capacité mémoire devient insuffisante.

Arya et al. [3] étudient sur NVIDIA Jetson AGX Orin l'effet de plusieurs paramètres, notamment la longueur des séquences, la taille du *batch* et la quantification. Ils observent que l'allongement des séquences réduit le débit et qu'une réduction de précision ne conduit pas systématiquement à une accélération, certains des plus petits modèles étudiés pouvant au contraire être ralentis.

Laskaridis et al. [14] et Abstreiter et al. [1] étudient également les compromis entre performances, mémoire et énergie lors de l'inférence locale sur différents matériels et modèles.

Giedra et al. [10] évaluent douze modèles allant de moins d'un milliard à environ huit milliards de paramètres sur une NVIDIA Jetson Orin Nano Super. Ils obtiennent notamment un débit médian de 18,13 tokens/s avec accélération GPU contre 7,12 tokens/s en exécution CPU seule.

Ces travaux confirment que le modèle, le mode d'exécution et les capacités de la plateforme influencent directement les performances obtenues sur *Edge*.

Parmi les travaux identifiés, Park et al. [21] se rapprochent davantage d'un contexte combinant *Edge* et concurrence. Les auteurs évaluent 21 moteurs sur plusieurs plateformes, allant d'infrastructures serveur à une NVIDIA Jetson AGX Orin, tout en faisant varier les longueurs d'entrée et de sortie ainsi que le niveau de concurrence. Leur campagne montre que les performances relatives des moteurs dépendent de la charge et de la plateforme considérée. Elle met toutefois également en évidence une difficulté de cette comparaison : tous les moteurs étudiés ne peuvent pas être déployés sur l'ensemble des matériels. Les auteurs ne parviennent notamment pas à exécuter vLLM sur la Jetson AGX Orin de leur campagne, de sorte que la comparaison réalisée sur *Edge* ne couvre pas les mêmes moteurs que celle menée sur les infrastructures serveur.

2.3.3 Synthèse et positionnement

Les travaux présentés apportent ainsi des réponses complémentaires mais sur des périmètres différents. Les études menées en environnement serveur montrent l'influence de la concurrence, de la charge des requêtes, du *batching* et de la gestion du cache KV sur les performances. Les caractérisations sur *Edge* montrent quant à elles que la plateforme, le modèle, la quantification et les ressources matérielles peuvent modifier le comportement de l'inférence. Park et al. [21] considèrent simultanément la plateforme *Edge* et la concurrence, mais ne peuvent pas évaluer les mêmes moteurs dans leurs environnements serveur et *Edge*.

Le Tableau 2.2 synthétise les dimensions couvertes par les principaux travaux de caractérisation présentés. Une croix indique qu'une dimension est explicitement étudiée dans le travail considéré.

À notre connaissance, les travaux identifiés ne permettent donc pas de caractériser conjointement, dans les mêmes conditions *Edge* multi-utilisateur, les performances côté utilisateur et les ressources mobilisées par des moteurs reposant sur des choix de conception différents.

Pour étudier cette intersection, nous retenons deux moteurs capables de traiter plusieurs requêtes sur les plateformes *Edge* considérées, mais issus d'orientations différentes. vLLM [13] est conçu pour le *serving* de requêtes concurrentes et intègre notamment PagedAttention, l'ordonnancement des requêtes et le *batching* continu. Bien qu'il soit principalement étudié sur des infrastructures serveur dans la littérature présentée précédemment, il peut être déployé sur les plateformes NVIDIA Jetson Orin utilisées dans notre étude. `llama.cpp` [9, 21] représente au contraire une approche légère et portable, particulièrement adaptée à l'exécution locale et largement utilisée dans les travaux sur *Edge*. Son mode serveur permet néanmoins de maintenir plusieurs séquences actives et de traiter plusieurs requêtes concurrentes. Les deux moteurs sont donc comparables dans un même scénario *Edge* multi-utilisateur, tout en reposant sur des priorités de conception différentes : vLLM

TABLE 2.2 – Dimensions couvertes par les travaux de caractérisation étudiés.

Travail	**Contexte**		**Facteurs étudiés**				**Mesures**	
	Edge	**Multi-utilisateur**	**Longueurs entrée/sortie**	**Concurrence**	**Modèle / quantification**	**Plateforme**	**Performance utilisateur**	**Ressources système**
Ay et al. [4]		X		X			X	X
Bomhold et al. [5]		X	X	X		X	X	X
Ye et al. [24]		X		X			X	
Dhar et al. [7]	X					X	X	X
Arya et al. [3]	X		X		X		X	X
Laskaridis et al. [14]	X		X		X	X	X	X
Abstreiter et al. [1]	X				X	X	X	X
Giedra et al. [10]	X				X		X	X
Park et al. [21]*	X	X	X	X	X	X	X	
Ce travail	X	X	X	X	X	X	X	X

* Park et al. [21] considèrent simultanément une plateforme *Edge* et la concurrence, mais tous les moteurs ne sont pas évalués dans les mêmes environnements. vLLM n'a notamment pas pu être déployé sur la NVIDIA Jetson AGX Orin utilisée dans leur campagne.

privilégie les mécanismes destinés au *serving* concurrent, tandis que `llama.cpp` privilégie une exécution locale légère et portable.

Cette comparaison vise ainsi à déterminer comment ces différences se traduisent lorsque les deux moteurs sont soumis aux mêmes modèles et aux mêmes charges sur une plateforme aux ressources limitées. Les facteurs mis en évidence dans la littérature, notamment la charge de travail, la concurrence, le modèle, la quantification et la plateforme, permettent de structurer une partie des questions de recherche. Leur étude est complétée par le partage des ressources, moins couvert dans les travaux présentés mais particulièrement pertinent dans un environnement *Edge* où le service LLM peut coexister avec d'autres applications. Ces dimensions guident les questions de recherche et la méthodologie expérimentale présentées dans le chapitre suivant.

2.4 Conclusion

Ce chapitre a présenté les principales solutions pour l'inférence multi-utilisateur et sur *Edge*, ainsi que les travaux consacrés à leur caractérisation. Cette analyse a permis de justifier le choix de `llama.cpp` et vLLM et de préciser le cadre de l'étude menée dans ce mémoire.

Le chapitre suivant présente la démarche expérimentale adoptée pour cette étude, ainsi que les plateformes, les configurations et le protocole considérés.

Chapitre 3

Méthodologie expérimentale

3.1 Introduction

Le chapitre précédent a permis de positionner l'étude dans la littérature et de justifier la comparaison de `llama.cpp` et vLLM dans un contexte *Edge* multi-utilisateur.

Afin de conduire cette comparaison de manière structurée, ce chapitre présente la méthodologie expérimentale retenue pour caractériser leur comportement. Les questions de recherche guidant l'étude sont d'abord introduites, puis le cadre expérimental commun est présenté à travers les plateformes utilisées, les configurations retenues pour les deux moteurs, les métriques collectées et le protocole appliqué aux différentes campagnes.

3.2 Questions de recherche

L'état de l'art a mis en évidence plusieurs facteurs influençant le comportement de l'inférence LLM. Les travaux menés en environnement multi-utilisateur montrent notamment l'effet de la concurrence et de la charge des requêtes [4, 5, 24], tandis que les caractérisations sur *Edge* soulignent l'influence du modèle, de la quantification et de la plateforme matérielle [7, 3, 1, 10]. Ces dimensions restent toutefois principalement étudiées séparément et rarement comparées, dans les mêmes conditions, sur des moteurs reposant sur des objectifs de conception différents. Notre caractérisation reprend donc progressivement ces facteurs dans un cadre expérimental commun : l'effet de la charge de travail est d'abord étudié, puis celui de la concurrence et du partage des ressources avec une autre application, dimension moins couverte dans les travaux présentés mais particulièrement pertinente sur *Edge*. Les tendances observées sont ensuite examinées sur d'autres modèles, quantifications et plateformes. Cette démarche conduit aux cinq questions de recherche suivantes :

— **Q1 : Influence de la charge de travail :** Comment les longueurs d'entrée et de sortie influencent-elles les performances, l'utilisation des ressources et la consommation énergétique de `llama.cpp` et de vLLM ?

— **Q2 : Influence de la concurrence :** Comment le comportement des deux moteurs évolue-t-il lorsque le nombre de requêtes concurrentes augmente ?

— **Q3 : Partage des ressources :** Comment le partage des ressources de la plateforme avec une autre application modifie-t-il le comportement des deux moteurs ?

— **Q4 – Généralisation à d'autres modèles et quantifications :** Les tendances observées se maintiennent-elles lorsque l'architecture, la taille ou la quantification

du modèle changent ?

— **Q5 – Généralisation à une autre plateforme *Edge* :** Les tendances obtenues sur la plateforme principale se maintiennent-elles sur une plateforme *Edge* disposant de ressources plus limitées ?

3.3 Cadre expérimental commun

Bien que le facteur étudié évolue d'une question à l'autre, les expériences reposent sur un cadre commun afin que les variations mesurées puissent être attribuées autant que possible au paramètre effectivement modifié. Ce cadre définit les plateformes utilisées, puis la manière dont une configuration de référence comparable a été construite pour `llama.cpp` et vLLM. Il précise ensuite les métriques collectées ainsi que le protocole appliqué aux différentes campagnes.

3.3.1 Plateformes expérimentales

Les expériences sont réalisées sur deux plateformes NVIDIA Jetson de la famille Orin. La Jetson AGX Orin 64 Go constitue la plateforme principale de l'étude, tandis que la Jetson Orin Nano 8 Go est utilisée dans Q5 afin de vérifier si les tendances observées se maintiennent sur une plateforme disposant de ressources plus limitées. Leurs principales caractéristiques sont présentées dans le Tableau 3.1.

TABLE 3.1 – Caractéristiques des plateformes expérimentales.

Caractéristique	**Jetson AGX Orin**	**Jetson Orin Nano**
CPU	12 cœurs ARM Cortex-A78AE	6 cœurs ARM Cortex-A78AE
Architecture GPU	NVIDIA Ampere	NVIDIA Ampere
Cœurs CUDA	2048	1024
Tensor Cores	64	32
Mémoire	64 Go LPDDR5	8 Go LPDDR5
Organisation mémoire	Mémoire unifiée CPU/GPU	Mémoire unifiée CPU/GPU
JetPack	6.2.2	6.2.2
Utilisation dans l'étude	Q1, Q2, Q3 et Q4	Q5

3.3.2 Configurations des moteurs

Afin de garantir la reproductibilité et la comparabilité des expériences, une configuration de référence commune est définie pour les deux moteurs. Elle est résumée dans le Tableau 3.2.

Les principaux choix associés à cette configuration sont les suivants :

— **Versions des moteurs :** les versions sont figées pour l'ensemble des campagnes afin d'éviter qu'une mise à jour ne modifie leur comportement entre les expériences. Le *build* `b9114` est retenu pour `llama.cpp`, tandis que vLLM 0.19.0 correspond à

TABLE 3.2 – Configurations de référence de `llama.cpp` et vLLM.

Paramètre	`llama.cpp`	**vLLM**
Version	`b9114`	0.19.0
Modèle	Qwen2.5-1.5B-Instruct	Qwen2.5-1.5B-Instruct
Quantification	Q8_0	W8A8
Séquences actives maximales	4	4
Taille du contexte	8192 *tokens* au total	2048 *tokens* par séquence
Prefix caching	Désactivé	Désactivé
Exécution du modèle	GPU	GPU
`gpu-memory-utilization`	–	0,70

la version fournie dans l'environnement Docker NVIDIA AI-IOT utilisé sur Jetson Orin.

— **Modèle et quantification :** la configuration de référence utilise Qwen2.5-1.5B-Instruct, famille régulièrement considérée dans les travaux consacrés à l'inférence sur *Edge* [1, 3]. Le même modèle de base est utilisé avec une représentation 8 bits prise en charge par chaque moteur : Q8_0 pour `llama.cpp` et W8A8 pour vLLM.

— **Séquences actives :** leur nombre maximal est fixé à quatre, valeur correspondant au parallélisme retenu automatiquement par `llama.cpp` dans la version utilisée et cohérente avec un contexte *Edge* [21]. La même limite est imposée à vLLM afin de comparer les moteurs avec une capacité identique, à l'aide de `-parallel 4` pour `llama.cpp` et de `-max-num-seqs 4` pour vLLM.

— **Taille du contexte :** une longueur maximale de 2048 *tokens*, comprenant le *prompt* et les *tokens* générés, est retenue par séquence. Elle est fixée directement avec `-max-model-len 2048` dans vLLM. Dans `llama.cpp`, les quatre séquences partagent le cache KV ; `-ctx-size` est donc fixé à $4 \times 2048 = 8192$ *tokens* afin de conserver une capacité équivalente.

— **Réutilisation des préfixes :** le *prefix caching* est désactivé dans les deux moteurs afin qu'une requête ne bénéficie pas du calcul réalisé lors d'une exécution précédente d'un même préfixe.

— **Mémoire de vLLM :** par défaut, vLLM préalloue jusqu'à 90 % de la mémoire disponible, ce qui s'est révélé inadapté aux plateformes Jetson où la mémoire est partagée entre le CPU et le GPU. Des essais préliminaires ont conduit à fixer `-gpu-memory-utilization` à 0,70 sur l'AGX Orin afin de préserver une marge suffisante pour le système tout en permettant l'exécution du moteur.

Au-delà de ces paramètres nécessaires au déploiement et à la comparabilité, aucun réglage spécifique n'est appliqué pour optimiser individuellement l'un des moteurs, afin de conserver autant que possible leur comportement propre.

3.3.3 Métriques et instrumentation

Une fois les configurations des deux moteurs définies, leur comparaison est réalisée selon deux niveaux d'observation complémentaires : les performances perçues par le client et les ressources mobilisées sur la plateforme. Les premières permettent d'évaluer le service rendu à l'utilisateur, tandis que les secondes permettent d'analyser la manière dont chaque moteur exploite les ressources matérielles disponibles.

Métriques côté client

Les performances sont mesurées directement du point de vue du client. Pour chaque requête, trois instants sont considérés : son lancement $t_{\text{début}}$, la réception du premier *token* t_{premier} et la fin de la réponse t_{fin}. À partir de ces instants, quatre métriques sont retenues :

— **(*Time To First Token* (TTFT)) :** il correspond au délai entre le lancement de la requête et la réception du premier *token* [21] :

$$TTFT = t_{\text{premier}} - t_{\text{début}}. \tag{3.1}$$

Cette métrique reflète principalement le délai perçu par l'utilisateur avant le début de la réponse.

— **Latence :** elle représente le temps total entre le lancement de la requête et la réception complète de la réponse [21] :

$$T_{\text{latence}} = t_{\text{fin}} - t_{\text{début}}. \tag{3.2}$$

Comme le TTFT, elle est mesurée de bout en bout et inclut donc l'éventuel temps d'attente introduit par le moteur lorsque plusieurs requêtes sont concurrentes.

— **(*Time Per Output Token* (TPOT)) :** il caractérise la vitesse de génération après la réception du premier *token* en calculant le temps moyen associé aux *tokens* suivants [21] :

$$TPOT = \frac{T_{\text{latence}} - TTFT}{N_{\text{sortie}} - 1}, \tag{3.3}$$

où N_{sortie} désigne le nombre de *tokens* générés par la requête.

— **Débit :** il mesure la capacité globale du moteur à produire des *tokens* lorsque plusieurs requêtes sont soumises [21]. Pour un niveau de concurrence C, il est calculé sur l'ensemble du *run* comme le rapport entre le nombre total de *tokens* générés et l'intervalle compris entre le début de la première requête et la fin de la dernière :

$$D = \frac{\sum_{i=1}^{C} N_{\text{sortie},i}}{\max_i(t_{\text{fin},i}) - \min_i(t_{\text{début},i})}. \tag{3.4}$$

Ces quatre métriques permettent ainsi de distinguer le délai avant le début de la réponse, la durée totale d'une requête, la vitesse de génération et la capacité du moteur à traiter une charge concurrente.

Métriques système et ressources

Les performances côté client sont complétées par des mesures réalisées sur la plateforme afin de caractériser les ressources mobilisées par chaque moteur. L'analyse porte sur l'utilisation du GPU et du CPU, l'empreinte mémoire ainsi que la consommation énergétique.

— **Utilisation du GPU :** elle correspond à la proportion de temps pendant laquelle le GPU est actif au cours de la fenêtre d'exécution et est exprimée en pourcentage. Elle permet d'observer dans quelle mesure chaque moteur exploite l'accélérateur graphique. Cette métrique est collectée avec `tegrastats` [1].

— **Utilisation du CPU :** elle représente la fraction de la capacité de calcul CPU utilisée pendant l'exécution et est également exprimée en pourcentage. Une mesure globale est retenue afin de prendre en compte l'activité CPU engendrée par l'ensemble du moteur et de ses différents processus. Elle est collectée à l'échelle du système à l'aide de la bibliothèque `psutil` [2].

— **Mémoire unifiée :** elle correspond à la quantité totale de mémoire utilisée sur la plateforme pendant l'exécution et est obtenue à partir de `tegrastats`. Sur les plateformes Jetson, cette mémoire est partagée entre le CPU et le GPU ; la mesure inclut donc les allocations des moteurs, mais également le système d'exploitation et les autres services actifs. Elle ne représente ainsi pas l'empreinte exclusive de chaque moteur, mais permet de comparer leur occupation globale de la mémoire dans un environnement système identique.

— **Consommation énergétique :** la puissance instantanée $P(t)$, exprimée en watts, est obtenue à partir des capteurs de puissance intégrés aux plateformes Jetson et exposés par `tegrastats`. À partir de ces mesures, l'énergie totale consommée pendant un *run* est obtenue par intégration de la puissance entre le début de la première requête et la fin de la dernière :

$$E_{\text{total}} = \int_{t_{\text{début}}}^{t_{\text{fin}}} P(t)\, dt. \tag{3.5}$$

Une puissance de référence P_0, mesurée lorsque le moteur est chargé mais qu'aucune requête n'est en cours d'exécution, permet également d'estimer l'énergie dynamique associée au traitement des requêtes :

$$E_{\text{dynamique}} = \int_{t_{\text{début}}}^{t_{\text{fin}}} \left(P(t) - P_0\right)\, dt. \tag{3.6}$$

La méthode utilisée pour mesurer P_0 ainsi que le déroulement des différentes phases précédant chaque *run* sont précisés dans la Section 3.3.4. Pour les campagnes nécessitant une observation détaillée des ressources, les mesures issues de `tegrastats`, notamment l'utilisation du GPU, la mémoire unifiée et la puissance, sont collectées toutes les 50 ms, tandis que les métriques système sont échantillonnées toutes les 100 ms.

1. `tegrastats` est un outil fourni par NVIDIA sur les plateformes Jetson permettant de suivre en temps réel plusieurs informations matérielles, notamment l'utilisation du GPU, la mémoire, les températures et la puissance.

2. `psutil` est une bibliothèque Python permettant d'accéder aux informations relatives à l'utilisation des ressources du système et des processus, notamment le CPU et la mémoire.

3.3.4 Protocole expérimental

Afin de limiter l'influence de l'état initial de la plateforme et de rendre les mesures comparables entre les différentes configurations, les campagnes reposent sur un même protocole d'exécution. Chaque configuration est répétée cinq fois et chaque répétition constitue un *run*. Les principales étapes de ce protocole sont détaillées ci-dessous :

1. **Démarrage du moteur :** le serveur correspondant au moteur étudié est lancé avec la configuration définie précédemment. Le *pipeline* attend ensuite que celui-ci soit disponible avant de poursuivre, de manière à exclure du *run* le chargement du modèle et l'initialisation du serveur.
2. **Stabilisation initiale :** une période de 30 s est respectée une fois le moteur disponible avant de lancer la première inférence. Elle permet de séparer le démarrage du serveur des étapes suivantes du protocole.
3. ***Warm-up* :** Une exécution de *warm-up* est réalisée avant les mesures afin de limiter les effets propres à la première exécution susceptibles d'affecter les résultats, ces effets avaient été identifiés lors des essais préliminaires. Ce *warm-up* n'est donc pas inclus dans l'analyse.
4. **Stabilisation avant mesure :** Le *warm-up* puis les *runs* successifs modifient temporairement l'état de la plateforme, notamment sa température, une attente de 150 s est donc appliquée entre les *runs* de `llama.cpp`, de 60 s entre ceux de vLLM et de 300 s entre les deux moteurs. Ces durées ont été retenues à partir d'essais préliminaires durant lesquels la température et l'utilisation du GPU ont principalement été suivies afin d'identifier un temps d'attente suffisant pour que la plateforme revienne vers un état suffisamment comparable avant la mesure suivante.

   Les 30 dernières secondes de chaque période de stabilisation sont utilisées pour mesurer la puissance de référence P_0 définie dans la Section 3.3.3.
5. **Soumission de la charge :** Une fois la plateforme stabilisée, la charge à exécuter est définie par la longueur du *prompt* P, le nombre de *tokens* à générer O et le niveau de concurrence C. Sa construction et sa soumission suivent alors les étapes suivantes :

   (a) les *prompts* sont d'abord générés côté client de manière à respecter la longueur P demandée,
   (b) pour un niveau de concurrence C, un *thread* est créé pour chaque requête et reçoit l'un de ces *prompts* ainsi que la consigne de générer O *tokens*,
   (c) une fois prêts, les *threads* se synchronisent sur une barrière commune, dont la libération déclenche l'envoi des requêtes au moteur aussi simultanément que possible.

   les mêmes *prompts* et les mêmes paramètres de génération sont ensuite réutilisés pour l'autre moteur afin de conserver une charge applicative identique lors de la comparaison.
6. **Collecte des mesures :** dès l'envoi des requêtes, les mesures sont collectées en parallèle côté client et côté système. Pour chaque requête, le client enregistre son instant de départ, l'arrivée du premier *token* et la fin de la réponse, à partir desquels sont calculés le TTFT, la latence, le TPOT et le débit. Pendant ce même intervalle, les moniteurs système enregistrent l'utilisation du GPU, du CPU, de la mémoire ainsi que la puissance.

7. **Fin du *run* et répétitions :** lorsque toutes les requêtes sont terminées, les mesures sont sauvegardées et le *run* prend fin. Si d'autres répétitions restent à exécuter, la période de stabilisation définie précédemment est appliquée avant de lancer le *run* suivant. Une fois l'ensemble des expériences d'un moteur terminé, celui-ci est arrêté avant de passer au second moteur, en respectant la période d'attente définie précédemment.

Les résultats obtenus sur les cinq répétitions sont ensuite agrégés. Les configurations propres à chaque question de recherche ainsi que les résultats correspondants sont détaillés dans le chapitre suivant, organisé successivement selon Q1 à Q5.

3.4 Conclusion

Ce chapitre a défini le cadre méthodologique utilisé pour répondre aux cinq questions de recherche de l'étude. Les deux moteurs sont évalués sur des plateformes et des charges contrôlées, à partir d'un ensemble commun de métriques côté client et système, et selon un protocole visant à assurer la comparabilité et la répétabilité des mesures.

Le chapitre suivant détaille les campagnes associées à chacune des questions de recherche, puis présente les résultats obtenus et leur analyse.

Chapitre 4

Campagnes expérimentales, résultats et analyse

4.1 Introduction

Le chapitre précédent a défini le cadre expérimental commun ainsi que les cinq questions de recherche guidant l'étude. Ce chapitre présente les campagnes menées pour y répondre et analyse les résultats obtenus selon la progression définie par ces questions, depuis l'influence de la charge de travail jusqu'à la généralisation sur une seconde plateforme *Edge*. Enfin, une synthèse met en relation les principaux comportements observés au cours de ces différentes campagnes.

4.2 Influence de la charge de travail

La première campagne cherche à déterminer comment la charge associée à une requête influence les performances, les ressources mobilisées et la consommation énergétique de `llama.cpp` et vLLM. Cette charge peut augmenter de deux manières : par l'allongement du *prompt*, qui accroît le travail réalisé pendant le *prefill*, ou par l'augmentation du nombre de *tokens* générés, qui multiplie les itérations successives de *decode*. Comme ces deux phases reposent sur des modes d'exécution différents, présentés respectivement dans les Sections 1.2.2 et 1.2.2, elles sont étudiées séparément afin de distinguer leurs effets sur les deux moteurs.

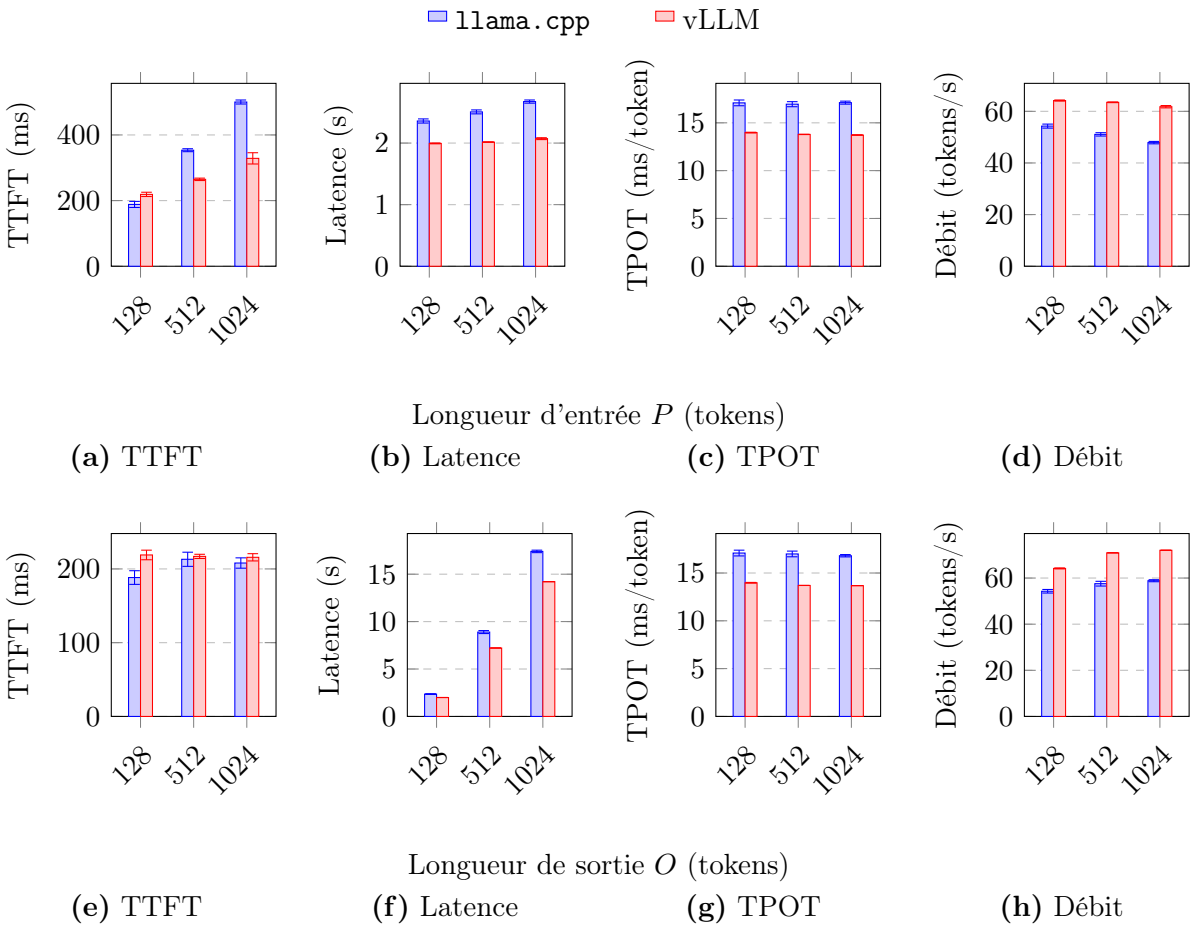
La concurrence est fixée à $C = 1$ afin d'isoler l'effet de la charge d'une requête, sans introduire les effets liés à l'exécution simultanée de plusieurs requêtes.

4.2.1 Configuration de l'expérience

Cette première campagne s'appuie sur le cadre expérimental défini dans le chapitre précédent, dont la configuration est conservée afin de ne faire varier que les paramètres liés à la charge de travail. Le profil $P128/O128$, correspondant à un *prompt* et une génération courts, est retenu comme référence, puis les longueurs 128, 512 et 1024 *tokens* sont considérées, soit une variation allant jusqu'à un facteur huit tout en restant compatible avec la limite de contexte de 2048 *tokens* fixée pour une requête. L'effet de la longueur d'entrée est ainsi étudié en maintenant $O = 128$ et en faisant varier P, tandis que l'effet de la longueur de sortie est étudié inversement en maintenant $P = 128$ et en faisant varier O.

4.2.2 Résultats et analyse

Les Figures 4.1 et 4.2 permettent de comparer les effets des longueurs d'entrée et de sortie selon deux dimensions complémentaires : les performances perçues côté utilisateur et les ressources mobilisées par la plateforme. Ces deux aspects sont analysés successivement ci-dessous.



(a) TTFT **(b)** Latence **(c)** TPOT **(d)** Débit

(e) TTFT **(f)** Latence **(g)** TPOT **(h)** Débit

FIGURE 4.1 – Influence des longueurs d'entrée et de sortie sur les performances côté client à $C = 1$.

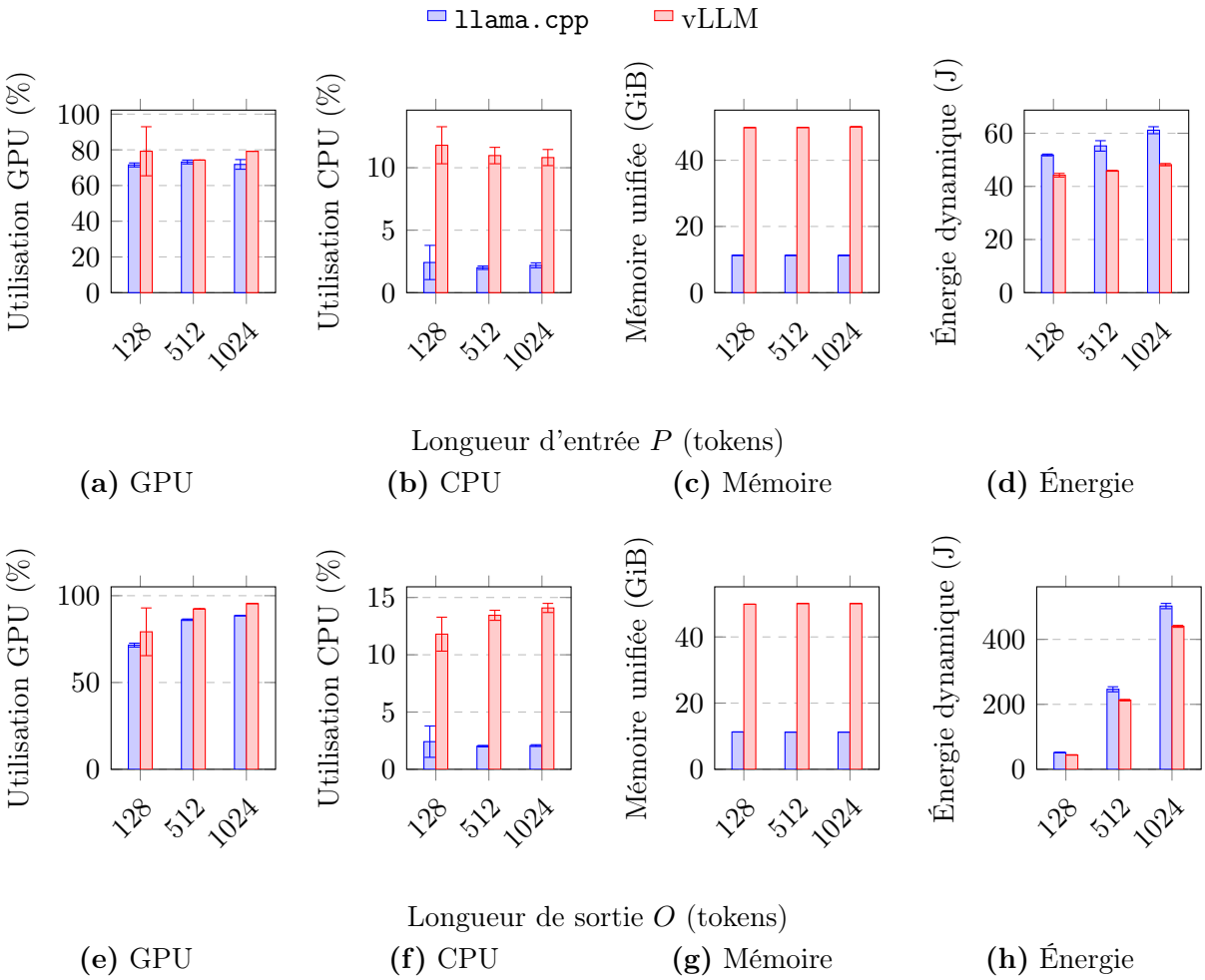
Performances. L'effet le plus net de la longueur d'entrée apparaît sur le TTFT (Figure 4.1(a)). Lorsque P passe de 128 à 1024 *tokens*, celui-ci augmente de 188 à 500 ms avec `llama.cpp`, contre 219 à 329 ms avec vLLM. L'avantage observé avec `llama.cpp` pour le *prompt* le plus court s'inverse ainsi dès $P = 512$, le TTFT de vLLM augmentant moins fortement lorsque le travail de *prefill* s'accroît. Une interprétation possible est liée au coût logiciel propre aux deux moteurs : dans cette campagne, `llama.cpp` s'exécute dans un processus unique, tandis que vLLM repose sur trois processus et sur des mécanismes supplémentaires d'ordonnancement et de coordination. Pour un *prompt* court, ces traitements peuvent représenter une part non négligeable du temps total et favoriser le chemin d'exécution plus léger de `llama.cpp` ; lorsque le *prefill* s'allonge, leur poids relatif diminue devant le coût du calcul lui-même, ce qui peut contribuer à l'inversion observée. Cette interprétation reste toutefois indicative, puisque la campagne ne permet pas d'isoler un mécanisme unique. À l'inverse, l'augmentation de O ne modifie pratiquement pas le

TTFT (Figure 4.1(e)), puisque les *tokens* supplémentaires sont produits après le premier ; `llama.cpp` conserve alors un TTFT légèrement inférieur sur les trois longueurs de sortie testées.

La latence confirme cette distinction entre les deux dimensions de la charge (Figures 4.1(b) et (f)) : elle n'augmente que modérément avec P, puisque les 128 étapes de *decode* restent identiques, alors qu'elle croît fortement avec O à mesure que le nombre d'itérations de génération augmente, jusqu'à atteindre 17,4 s avec `llama.cpp` et 14,2 s avec vLLM pour $O = 1024$. Le TPOT reste en revanche globalement stable dans les deux expériences, autour de 17 ms/*token* avec `llama.cpp` et de 14 ms/*token* avec vLLM (Figures 4.1(c) et (g)). L'augmentation de la latence associée à une sortie longue provient donc principalement de la répétition d'un plus grand nombre d'itérations de *decode*, et non d'une hausse importante du coût moyen de chaque *token*. Le débit reflète directement ces deux comportements (Figures 4.1(d) et (h)). Lorsque P augmente, davantage de temps est consacré au *prefill* alors que le nombre de *tokens* générés reste fixé à 128, ce qui réduit le nombre moyen de *tokens* produits par seconde. Lorsque O augmente, le débit progresse au contraire légèrement : le coût initial de la requête, notamment le *prefill* et le temps précédant la production du premier *token*, est alors réparti sur un nombre plus important de *tokens* générés. Sur l'ensemble des profils, vLLM conserve ainsi un TPOT plus faible, un débit plus élevé et une latence plus courte.

Ressources et énergie. Les ressources moyennes sont beaucoup moins sensibles à la longueur d'entrée. L'utilisation GPU ne présente pas de progression monotone lorsque P augmente (Figure 4.2(a)), puisque le surcroît de calcul est concentré dans un *prefill* relativement court, puis moyenné avec le même *decode* de 128 *tokens*. Lorsque O augmente, le *decode* occupe au contraire une part croissante de la fenêtre de mesure et maintient le GPU actif plus longtemps, jusqu'à atteindre environ 89 % avec `llama.cpp` et 95 % avec vLLM pour $O = 1024$ (Figure 4.2(e)). L'utilisation CPU varie elle aussi peu avec la charge, mais l'écart entre les deux moteurs reste marqué, vLLM sollicitant davantage le CPU que `llama.cpp` (Figures 4.2(b) et (f)). Cet écart est cohérent avec l'observation précédente sur leur organisation logicielle : `llama.cpp` s'exécute avec un seul processus, tandis que vLLM en mobilise trois.

La mémoire unifiée totale reste également presque indépendante de P et de O, autour de 11,3 GiB avec `llama.cpp` et de 50 GiB avec vLLM (Figures 4.2(c) et (g)). Cette stabilité s'explique par des capacités principalement déterminées au démarrage : `llama.cpp` charge le modèle ainsi que les autres éléments nécessaires à son exécution, puis dimensionne notamment son cache KV à partir de `-ctx-size 8192`. vLLM utilise quant à lui le budget défini par `-gpu-memory-utilization 0.70` pour charger les éléments nécessaires à son exécution, la capacité restante de ce budget servant ensuite à dimensionner et préallouer le pool de blocs du cache KV, selon le principe de PagedAttention présenté dans la Section 1.3.2 [13]. Enfin, l'énergie dynamique augmente avec la quantité de travail exécutée (Figures 4.2(d) et (h)). L'effet reste modéré lorsque seul le *prompt* s'allonge : entre $P = 128$ et $P = 1024$, l'augmentation est d'environ 18 % avec `llama.cpp` et 9 % avec vLLM. Elle devient beaucoup plus forte avec la longueur de sortie, puisque chaque *token* supplémentaire ajoute une nouvelle itération de *decode* : lorsque O est multiplié par huit, l'énergie dynamique est multipliée par environ 9,7 avec `llama.cpp` et par 10 avec vLLM. Malgré une occupation CPU et mémoire plus importante, vLLM conserve ainsi, selon la charge, une énergie dynamique inférieure d'environ 12 à 21 %, principalement grâce à une exécution plus rapide.



(a) GPU **(b)** CPU **(c)** Mémoire **(d)** Énergie

(e) GPU **(f)** CPU **(g)** Mémoire **(h)** Énergie

FIGURE 4.2 – Influence des longueurs d'entrée et de sortie sur les ressources mobilisées et l'énergie dynamique à $C = 1$.

Ces résultats répondent à Q1 : l'allongement du *prompt* affecte principalement le TTFT, avec un avantage d'environ 14 % pour `llama.cpp` à $P = 128$, qui s'inverse lorsque P augmente jusqu'à atteindre environ 34 % en faveur de vLLM à $P = 1024$. L'allongement de la génération agit surtout sur la durée du *decode* : la latence et l'énergie augmentent fortement, tandis que le TPOT reste presque stable. Le compromis entre les deux moteurs reste donc clair : `llama.cpp` mobilise moins de CPU et de mémoire et conserve le meilleur TTFT pour le *prompt* le plus court, tandis que vLLM maintient, sur l'ensemble des profils, un TPOT et une latence plus faibles, un débit supérieur et une énergie dynamique inférieure de 12 à 21 %.

4.3 Influence de la concurrence

Après avoir caractérisé l'effet de la charge sur une requête isolée, cette deuxième campagne étudie le comportement de `llama.cpp` et de vLLM lorsque plusieurs requêtes sont soumises simultanément. L'objectif est de déterminer jusqu'à quel point l'augmentation de la concurrence accroît réellement la capacité de traitement, puis d'identifier les conséquences d'une demande supérieure au nombre de séquences que les moteurs peuvent maintenir actives en parallèle.

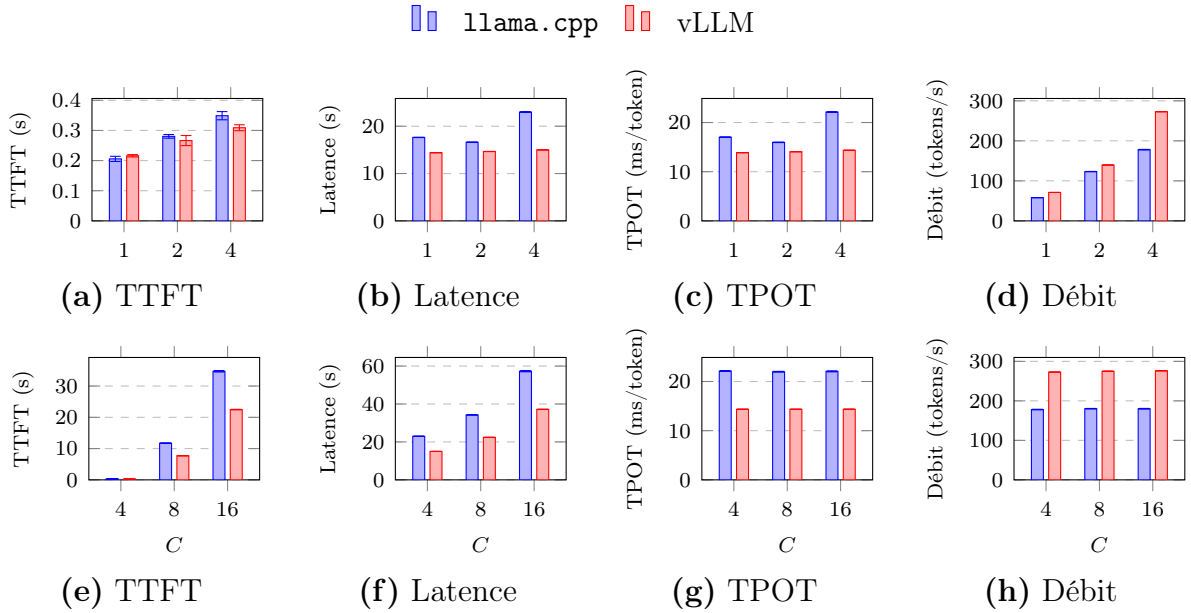
4.3.1 Configuration de l'expérience

Trois profils issus de Q1 sont retenus afin de vérifier que l'effet de la concurrence ne dépend pas d'un profil de charge particulier. Le profil $P128/O128$ constitue le cas léger de référence, tandis que $P1024/O128$ augmente principalement le travail de *prefill* en conservant une génération courte et que $P128/O1024$ maintient le même *prompt* court mais prolonge fortement le *decode*.

Pour chacun de ces profils, la concurrence prend les valeurs $C \in \{1, 2, 4, 8, 16\}$, de manière à couvrir les régimes situés de part et d'autre de la capacité active fixée à quatre séquences pour les deux moteurs. $C = 1$ fournit la référence sans concurrence, $C = 2$ introduit un premier niveau de parallélisme sans atteindre cette limite et $C = 4$ exploite l'ensemble des séquences disponibles. Les valeurs $C = 8$ et $C = 16$ soumettent ensuite respectivement deux et quatre fois plus de requêtes que de séquences pouvant être actives simultanément, afin d'étudier le passage d'un parallélisme utile à un régime de surcharge dans lequel certaines requêtes doivent attendre.

4.3.2 Résultats et analyse

L'exploration des trois profils met en évidence le même changement de régime : jusqu'à $C = 4$, l'augmentation de la concurrence permet d'accroître le nombre de requêtes effectivement traitées en parallèle, tandis qu'au-delà les requêtes supplémentaires sont mises en attente. L'analyse détaillée, présentée dans les Figures 4.3 et 4.4, se concentre sur $P128/O1024$, qui rend ce changement visible puisque son *decode* long maintient les séquences actives plus longtemps. Ce profil conserve également un *prompt* court avec $P = 128$, ce qui permet d'observer l'évolution du TTFT lorsque la concurrence augmente sans modifier la longueur d'entrée de chaque requête.



(a) TTFT (b) Latence (c) TPOT (d) Débit

(e) TTFT (f) Latence (g) TPOT (h) Débit

FIGURE 4.3 – Influence de la concurrence sur les performances côté client pour $P128/O1024$.

Performances. Jusqu'à $C = 4$, le TTFT augmente pour les deux moteurs, mais plus rapidement avec `llama.cpp`, de sorte que son léger avantage initial disparaît lorsque les

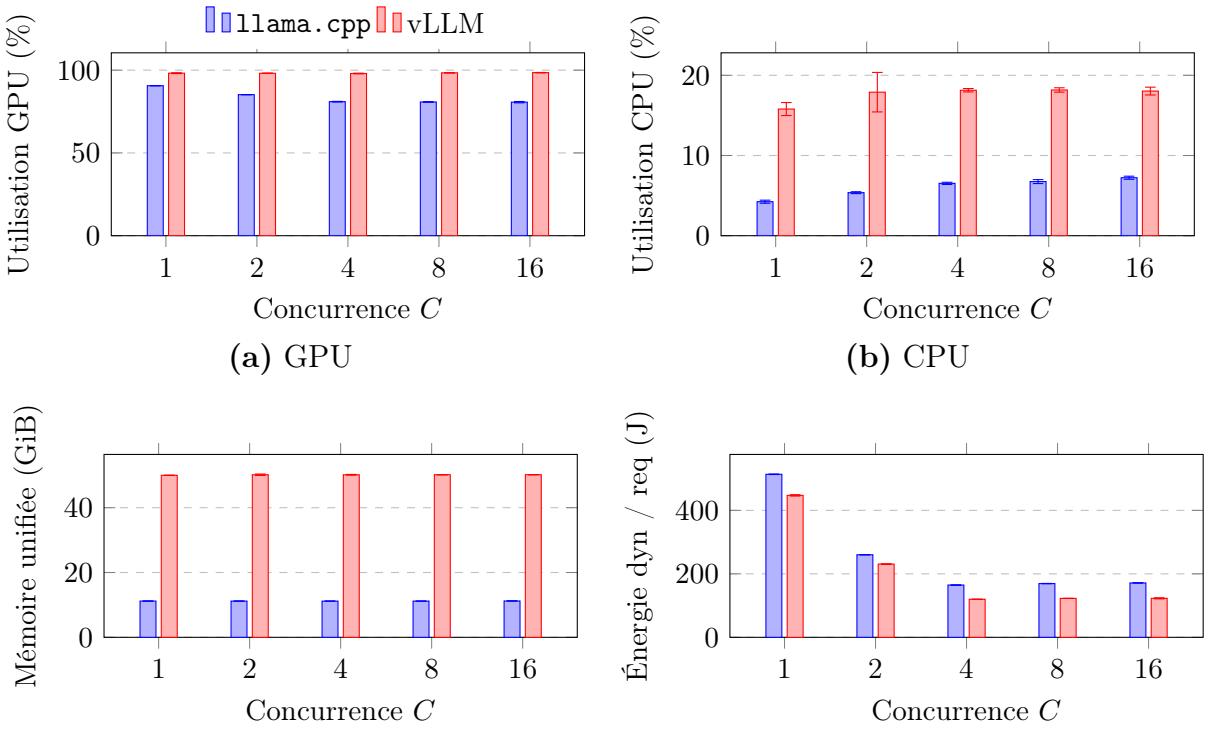
quatre séquences deviennent actives (Figure 4.3(a)). Cette évolution rappelle l'inversion observée en Q1 lorsque la longueur du *prompt* augmentait, mais le mécanisme n'est pas strictement équivalent : ici, chaque requête conserve un *prompt* de 128 *tokens*, tandis que l'augmentation de la concurrence accroît le volume agrégé de *prefill* à traiter simultanément et introduit les coûts liés à l'ordonnancement de plusieurs requêtes. La latence augmente également avec la concurrence, mais plus fortement avec `llama.cpp` (Figure 4.3(b)). Cet écart ne s'explique pas uniquement par l'augmentation du nombre de requêtes traitées en parallèle : le TPOT de `llama.cpp` se dégrade d'environ 30 % entre $C = 1$ et $C = 4$, alors que celui de vLLM reste pratiquement stable autour de 14 ms/*token* (Figure 4.3(c)). La génération elle-même devient donc plus lente avec `llama.cpp` lorsque plusieurs séquences sont actives, tandis que vLLM conserve presque le même coût moyen par *token*. Ce comportement suggère que `llama.cpp` maintient moins efficacement l'exécution du calcul sous concurrence ; les mesures de ressources permettront ensuite d'examiner plus précisément cette différence. Le débit augmente néanmoins fortement pour les deux moteurs jusqu'à $C = 4$, ce qui confirme que le parallélisme supplémentaire reste utile dans cette plage (Figure 4.3(d)). Le gain est toutefois plus proche de l'augmentation idéale avec vLLM : à $C = 4$, l'efficacité du parallélisme atteint environ 96 % contre 77 % avec `llama.cpp`. vLLM exploite donc plus efficacement les séquences actives, en augmentant davantage son débit tout en maintenant son TPOT presque constant.

Au-delà de $C = 4$, le comportement change nettement. Le TTFT augmente fortement pour les deux moteurs, puisque les requêtes supplémentaires ne peuvent plus devenir immédiatement actives et doivent attendre qu'une des quatre séquences se libère (Figure 4.3(e)). La latence suit la même évolution et se dégrade rapidement avec la concurrence (Figure 4.3(f)). Le TPOT reste en revanche pratiquement constant au-delà de cette limite, autour de 22 ms/*token* avec `llama.cpp` et de 14 ms/*token* avec vLLM (Figure 4.3(g)). Une fois qu'une requête devient active, sa vitesse de génération ne se dégrade donc plus sensiblement lorsque C continue d'augmenter. Dans le même temps, le débit plafonne presque complètement pour les deux moteurs (Figure 4.3(h)), ce qui montre que les requêtes supplémentaires n'augmentent plus la capacité de traitement mais prolongent principalement leur temps d'attente. Une génération longue amplifie cet effet puisqu'elle maintient les quatre séquences actives plus longtemps avant de libérer une place.

Ressources et énergie. L'utilisation du GPU évolue différemment avec la concurrence selon le moteur. Entre $C = 1$ et $C = 4$, elle diminue d'environ 91 % à 81 % avec `llama.cpp`, alors que vLLM maintient une occupation proche de 98 % (Figure 4.4(a)). Cette baisse accompagne la dégradation du TPOT observée précédemment avec `llama.cpp` et indique que l'augmentation du nombre de séquences actives ne se traduit pas par une occupation plus continue du GPU. Une analyse ciblée avec NVIDIA Nsight Systems [1] est donc utilisée pour examiner plus précisément l'enchaînement des travaux soumis au GPU.

La Figure 4.5 montre effectivement une exécution moins continue avec `llama.cpp` lorsque la concurrence augmente. La fraction de la fenêtre pendant laquelle au moins un *kernel* est actif passe de 82,0 % à $C = 1$ à 70,3 % à $C = 4$, tandis qu'elle reste proche de 98 % avec vLLM. Dans le même temps, la durée moyenne des intervalles d'inactivité entre deux périodes d'activité GPU augmente de 6,35 à 9,72 μs avec `llama.cpp`, tandis qu'elle reste inférieure à 0,4 μs avec vLLM. Les traces permettent d'éclairer cette

1. `Nsight Systems` est un outil de profilage système de NVIDIA permettant d'observer sur une chronologie l'activité du CPU et du GPU, notamment l'exécution des *kernels* CUDA, les appels CUDA effectués côté hôte et les phases de synchronisation.



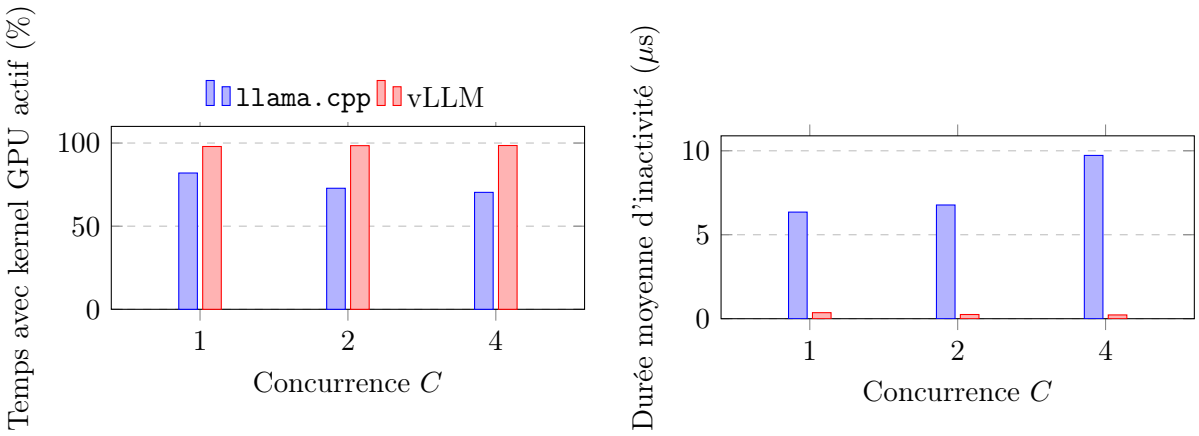
(a) GPU **(b)** CPU

(c) Mémoire unifiée **(d)** Énergie dynamique par requête

FIGURE 4.4 – Influence de la concurrence sur les ressources mobilisées et l'énergie dynamique pour $P128/O1024$.

différence : avec `llama.cpp`, les étapes successives de génération sont séparées par des synchronisations côté hôte avant la soumission de nouveaux travaux au GPU. Autrement dit, après le traitement d'un *token*, le GPU peut rester momentanément sans travail pendant que le côté hôte se synchronise et prépare la soumission suivante. Lorsque plusieurs séquences sont actives, ces interruptions subsistent et limitent donc la capacité du moteur à maintenir le GPU occupé en continu. Avec vLLM, les soumissions s'enchaînent au contraire de manière beaucoup plus continue et les traitements côté hôte peuvent se chevaucher davantage avec l'activité GPU, ce qui permet de maintenir l'accélérateur presque constamment actif. Cette différence est cohérente avec le maintien du TPOT de vLLM et avec sa meilleure efficacité du parallélisme jusqu'à $C = 4$. L'utilisation CPU augmente également jusqu'à $C = 4$, tout en restant plus élevée avec vLLM (Figure 4.4(b)). La mémoire unifiée varie quant à elle très peu avec la concurrence (Figure 4.4(c)), ce qui est cohérent avec les stratégies d'allocation décrites en Q1 : l'augmentation du nombre de séquences actives modifie principalement l'utilisation des capacités déjà réservées plutôt que la quantité globale de mémoire allouée. Enfin, l'énergie dynamique par requête diminue jusqu'à $C = 4$, puisque plusieurs requêtes partagent alors une même fenêtre d'exécution et que l'augmentation du débit compense le coût énergétique supplémentaire lié au parallélisme (Figure 4.4(d)). Elle passe ainsi d'environ 514 à 165 J avec `llama.cpp` et de 447 à 120 J avec vLLM. Au-delà de quatre séquences actives, elle évolue peu, car les requêtes supplémentaires restent principalement en attente sans produire de gain comparable en débit.

Ces résultats répondent à Q2 : jusqu'à $C = 4$, l'augmentation de la concurrence accroît réellement la capacité de traitement, mais plus efficacement avec vLLM, qui atteint environ 96 % de l'augmentation idéale du débit contre 77 % pour `llama.cpp`, tout en



(a) Continuité de l'activité GPU **(b)** Intervalles d'inactivité GPU

FIGURE 4.5 – Activité GPU de `llama.cpp` et vLLM sous concurrence pour $P128/O1024$.

maintenant un TPOT presque stable. Au-delà de quatre séquences actives, le débit plafonne pour les deux moteurs et les requêtes supplémentaires augmentent principalement le temps d'attente. Le compromis entre les deux moteurs reste donc clair : `llama.cpp` mobilise moins de CPU et de mémoire, tandis que vLLM maintient une activité GPU plus continue et conserve un TPOT et une latence plus faibles, un débit supérieur ainsi qu'une énergie dynamique par requête inférieure.

4.4 Influence du partage des ressources

Les deux premières campagnes ont étudié les moteurs lorsque les ressources de la plateforme étaient entièrement disponibles pour l'inférence LLM. Mais dans un environnement *Edge*, une application ne s'exécute pas nécessairement seule : plusieurs services peuvent coexister sur une même plateforme et se partager ses ressources.

Cette troisième campagne étudie donc le comportement de `llama.cpp` et de vLLM lorsqu'une application co-localisée sollicite simultanément le CPU et la mémoire de la plateforme. L'objectif est de déterminer dans quelle mesure ce partage des ressources affecte leurs performances et s'il modifie le compromis observé entre les deux moteurs dans les campagnes précédentes.

4.4.1 Configuration de l'expérience

Cette campagne s'appuie sur la configuration de référence utilisée dans les campagnes précédentes. Deux profils sont retenus : $P128/O128$, représentant une charge courte, et $P128/O1024$, qui prolonge le *decode*. La concurrence est fixée à $C = 4$, niveau auquel les quatre séquences disponibles sont exploitées sans introduire le régime de mise en attente identifié en Q2.

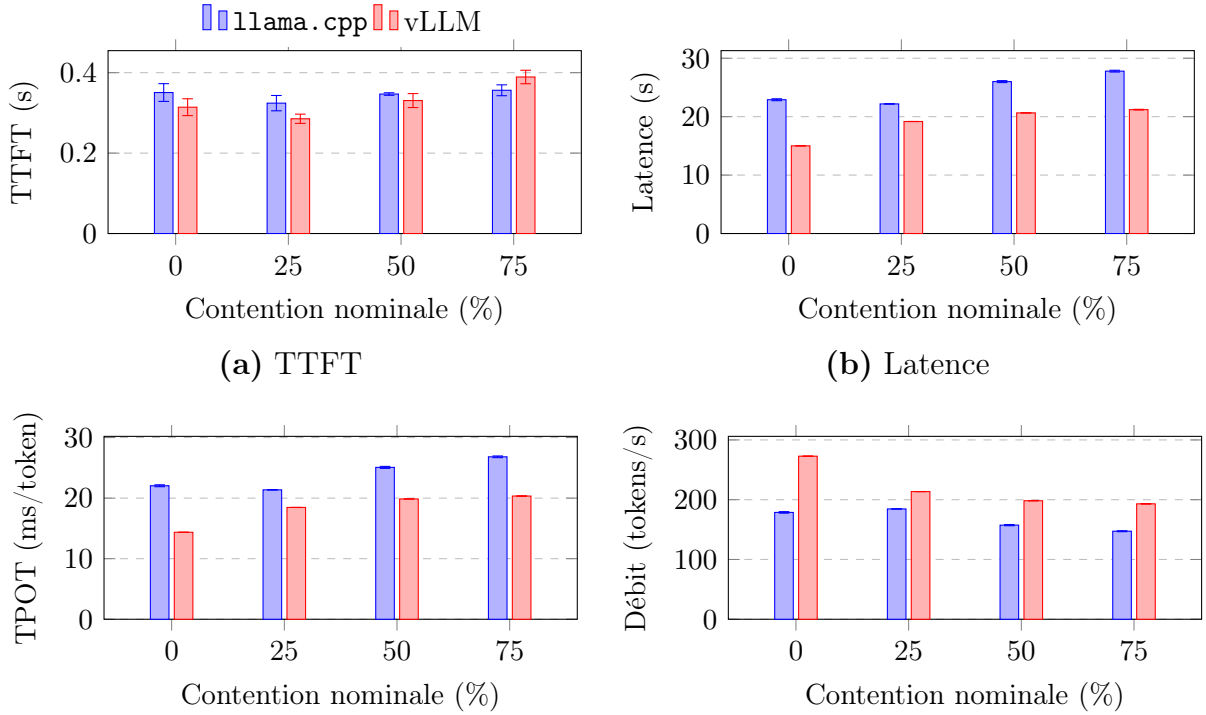
Le partage des ressources est reproduit au moyen d'une application synthétique exécutée en parallèle de l'inférence. Elle utilise plusieurs processus Python, chacun épinglé sur un cœur CPU. Chaque processus effectue en continu des additions et soustractions vectorielles sur trois tableaux de 128 MiB, afin de solliciter simultanément le CPU et la bande passante mémoire. Quatre niveaux de contention sont considérés : 0, 25, 50 et 75 %. Le niveau 0 % correspond à l'absence d'application concurrente, tandis que les trois

autres utilisent respectivement 3, 6 et 9 processus, soit autant de cœurs parmi les douze disponibles sur l'AGX Orin. La quantité de données manipulée simultanément atteint ainsi environ 1,125, 2,25 et 3,375 GiB.

Pour chaque niveau, la puissance de référence P_0 est mesurée alors que l'application concurrente est déjà active. L'énergie dynamique par requête correspond ainsi à l'énergie supplémentaire consommée lors de l'ajout de l'inférence LLM à cet état partagé.

4.4.2 Résultats et analyse

Les deux profils testés présentent les mêmes tendances lorsque le niveau de contention augmente. L'analyse détaillée, présentée dans les Figures 4.6 et 4.7, se concentre donc sur $P128/O1024$, dont le *decode* prolongé maintient l'inférence active plus longtemps et rend les effets du partage des ressources plus visibles.



(a) TTFT **(b)** Latence

(c) TPOT **(d)** Débit

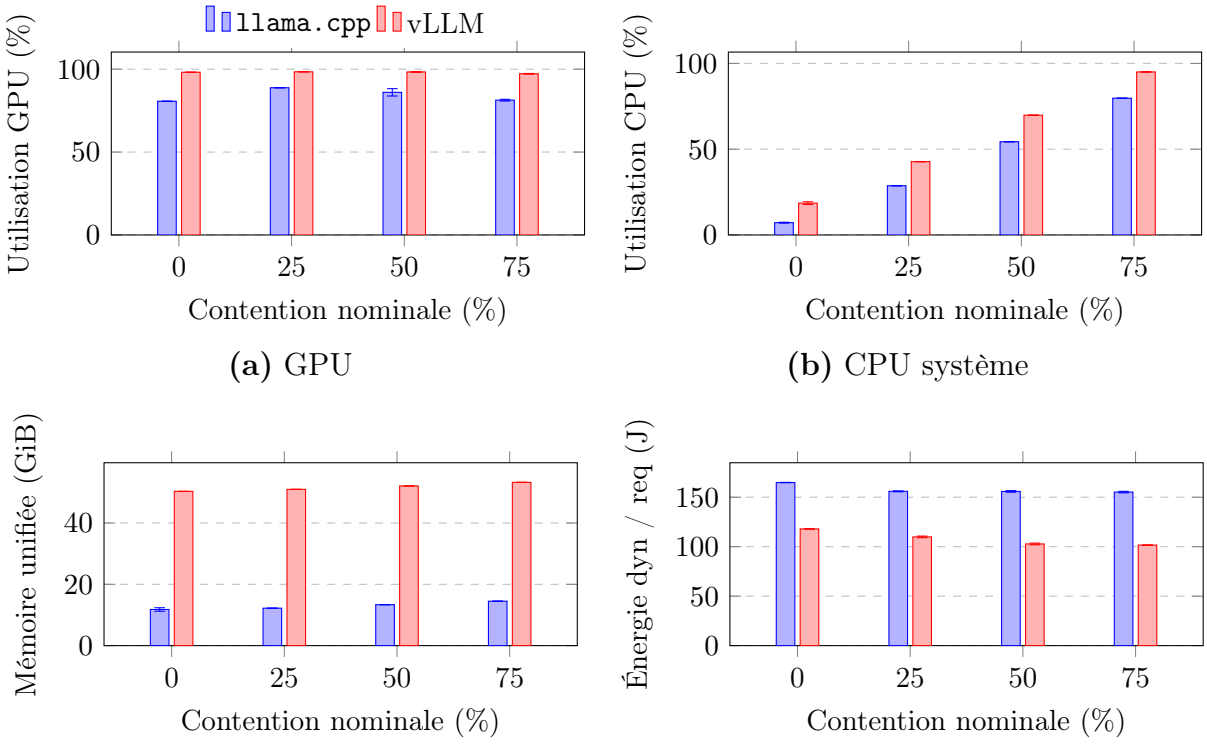
FIGURE 4.6 – Influence du partage des ressources sur les performances côté client pour $P128/O1024$ à $C = 4$.

Performances. Le TTFT évolue peu aux premiers niveaux de contention et diminue même légèrement à 25 % pour les deux moteurs, avant d'augmenter lorsque la pression sur les ressources devient plus forte (Figure 4.6(a)). À 75 %, le classement observé sans contention s'inverse : `llama.cpp` fournit alors le premier *token* environ 9 % plus rapidement que vLLM. Cette inversion montre que le TTFT de vLLM devient plus sensible lorsque les ressources sont fortement partagées.

La latence se dégrade également pour les deux moteurs, mais avec des amplitudes différentes (Figure 4.6(b)) : entre 0 et 75 % de contention, elle augmente d'environ 21 % avec `llama.cpp`, contre 41 % avec vLLM. vLLM reste néanmoins plus rapide sur l'ensemble des niveaux testés.

Cette dégradation plus marquée se retrouve pendant le *decode*. Le TPOT augmente d'environ 22 % avec `llama.cpp` et de 42 % avec vLLM sur la même plage (Figure 4.6(c)). vLLM est ainsi proportionnellement plus sensible à la pression exercée sur les ressources partagées, même s'il conserve un TPOT inférieur à celui de `llama.cpp`.

Cette sensibilité se retrouve également sur le débit (Figure 4.6(d)) : à 75 % de contention, la baisse atteint environ 18 % avec `llama.cpp`, contre 29 % avec vLLM. L'écart entre les moteurs se réduit donc progressivement, mais vLLM conserve un débit supérieur dans toutes les configurations testées. Le partage des ressources réduit ainsi son avantage sans inverser le classement en latence, en TPOT et en débit.



(a) GPU **(b)** CPU système

(c) Mémoire unifiée **(d)** Énergie dynamique par requête

FIGURE 4.7 – Influence du partage des ressources sur les ressources mobilisées et l'énergie dynamique pour $P128/O1024$ à $C = 4$.

Ressources et énergie. L'utilisation du GPU met en évidence un comportement différent entre les deux moteurs (Figure 4.7(a)). Avec `llama.cpp`, elle augmente d'abord au premier niveau de contention, en même temps que les performances s'améliorent légèrement, puis diminue lorsque la pression devient plus forte. vLLM maintient au contraire une activité GPU proche de 100 % sur toute la plage, alors même que son TPOT augmente et que son débit diminue nettement. Une forte occupation du GPU ne signifie donc pas nécessairement que l'inférence progresse à la même vitesse lorsque le CPU et la mémoire sont simultanément sollicités.

L'utilisation CPU augmente logiquement avec le nombre de processus concurrents, mais reste systématiquement plus élevée avec vLLM (Figure 4.7(b)). Au niveau de contention le plus élevé, le système atteint environ 95 % d'utilisation CPU avec vLLM, contre 80 % avec `llama.cpp`. vLLM dispose ainsi de moins de marge CPU lorsque la plateforme est partagée.

La mémoire utilisée augmente progressivement avec la contention en raison des tableaux manipulés par les processus concurrents (Figure 4.7(c)). Cette augmentation ne modifie toutefois pas le contraste observé précédemment : l'utilisation mémoire totale reste nettement plus faible avec `llama.cpp` qu'avec vLLM.

Enfin, l'énergie dynamique par requête diminue légèrement lorsque la contention augmente, tout en restant plus faible avec vLLM (Figure 4.7(d)). Cette évolution ne signifie pas que la contention améliore l'efficacité énergétique de l'inférence. Pour chaque niveau, P_0 est mesurée avec l'application concurrente déjà active ; la métrique représente donc uniquement l'énergie supplémentaire consommée lorsque le LLM est ajouté à cet état du système. Cette contribution peut ainsi diminuer alors même que le temps d'exécution augmente.

Ces résultats répondent à Q3 : le partage simultané du CPU et de la mémoire dégrade les performances des deux moteurs, mais vLLM est proportionnellement plus sensible à l'augmentation de la contention. À 75 %, le classement du TTFT s'inverse en faveur de `llama.cpp`, sans remettre en cause l'avantage de vLLM en latence, en TPOT, en débit et en énergie dynamique par requête. Le compromis entre les deux moteurs reste donc clair dans un environnement partagé : `llama.cpp` mobilise moins de CPU et de mémoire et se dégrade moins lorsque la contention augmente, tandis que vLLM conserve de meilleures performances globales malgré une sensibilité plus forte au partage des ressources.

4.5 Généralisation à d'autres modèles et quantifications

Les trois premières campagnes ont caractérisé les deux moteurs à partir du même modèle de référence, Qwen2.5-1.5B-Instruct. Cette quatrième campagne cherche à déterminer si les tendances mises en évidence jusqu'ici se maintiennent lorsque la taille du modèle, la famille de modèles considérée ou sa quantification changent.

4.5.1 Configuration de l'expérience

La concurrence est fixée à $C = 4$, afin d'exploiter les quatre séquences actives sans introduire le régime de mise en attente identifié en Q2. Les profils $P128/O128$ et $P128/O1024$ sont testés pour chacune des configurations considérées.

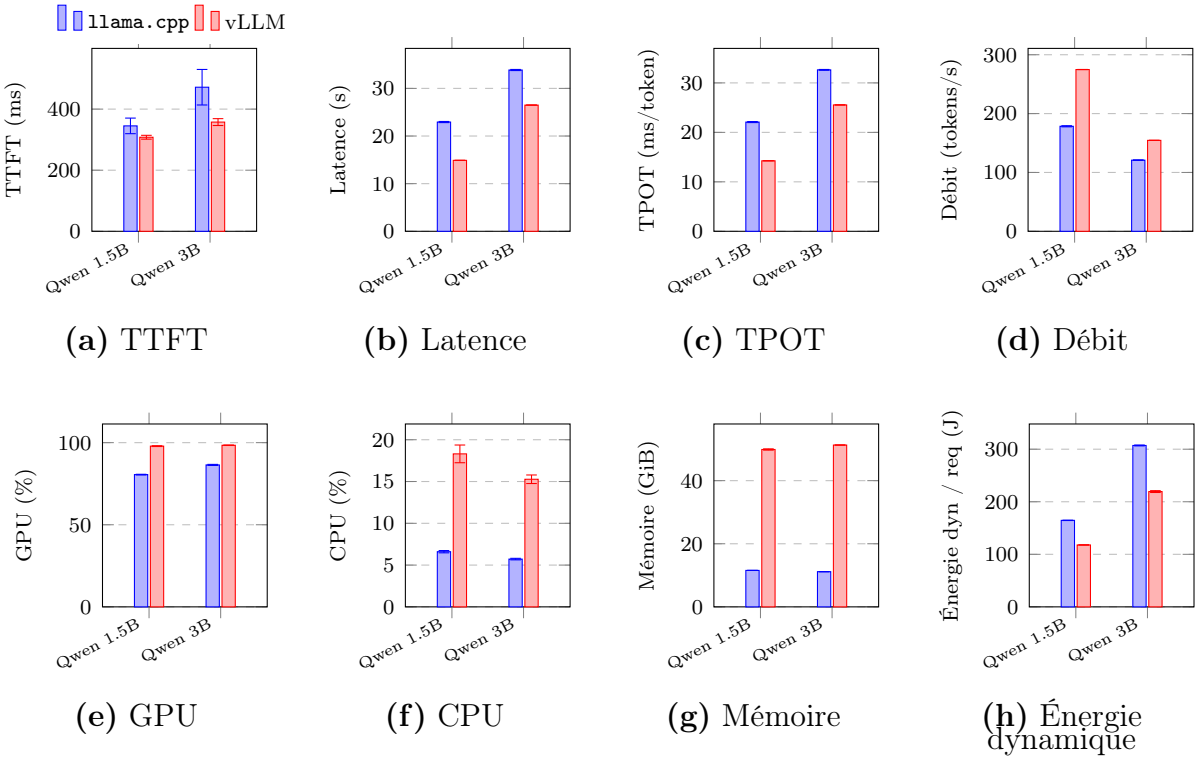
Trois variations sont étudiées séparément. L'effet de la taille est d'abord évalué au sein d'une même famille en comparant Qwen2.5-1.5B-Instruct à Qwen2.5-3B-Instruct, en conservant les configurations 8 bits de référence : Q8_0 avec `llama.cpp` et W8A8 avec vLLM. Le changement de modèle est ensuite étudié en comparant Qwen2.5-1.5B-Instruct à Llama 3.2-1B-Instruct. Les deux modèles n'ayant pas exactement le même nombre de paramètres, cette comparaison ne constitue pas une isolation stricte de l'architecture ; elle permet néanmoins de vérifier si les tendances observées avec Qwen2.5 se maintiennent avec une autre famille de modèles de taille voisine.

Enfin, l'effet de la quantification est étudié sur Qwen2.5-1.5B-Instruct. Pour `llama.cpp`, Q8_0 est comparé à Q4_K_M, tandis que pour vLLM, W8A8 est comparé à un *checkpoint* AWQ INT4 exécuté avec le *backend* AWQ-Marlin. Ces choix appartiennent à des familles de quantification représentatives de leurs écosystèmes respectifs : les K-types de `llama.cpp` sont notamment considérés dans des travaux récents sur l'inférence LLM en environnement *Edge* [6], Q4_K_M étant explicitement évalué sur plusieurs familles de modèles, dont Qwen et Llama [12]. AWQ est quant à lui une méthode de quantification des poids conçue pour la compression et l'accélération de l'inférence LLM [19]. Q4_K_M et

AWQ ne reposant pas sur la même méthode de quantification, leur comparaison ne vise pas à déterminer lequel des deux formats est intrinsèquement meilleur, mais à étudier l'effet pratique d'une réduction supplémentaire de précision dans chacun des moteurs.

4.5.2 Résultats et analyse

Les deux profils testés présentent les mêmes tendances pour les trois variations considérées. L'analyse détaillée se concentre donc sur $P128/O1024$. Les Figures 4.8, 4.9 et 4.10 présentent respectivement les résultats obtenus pour l'augmentation de la taille du modèle, le changement de famille de modèles et la quantification. Dans chaque figure, la première ligne regroupe les performances côté client, tandis que la seconde présente les ressources mobilisées et l'énergie consommée.



(a) TTFT **(b)** Latence **(c)** TPOT **(d)** Débit

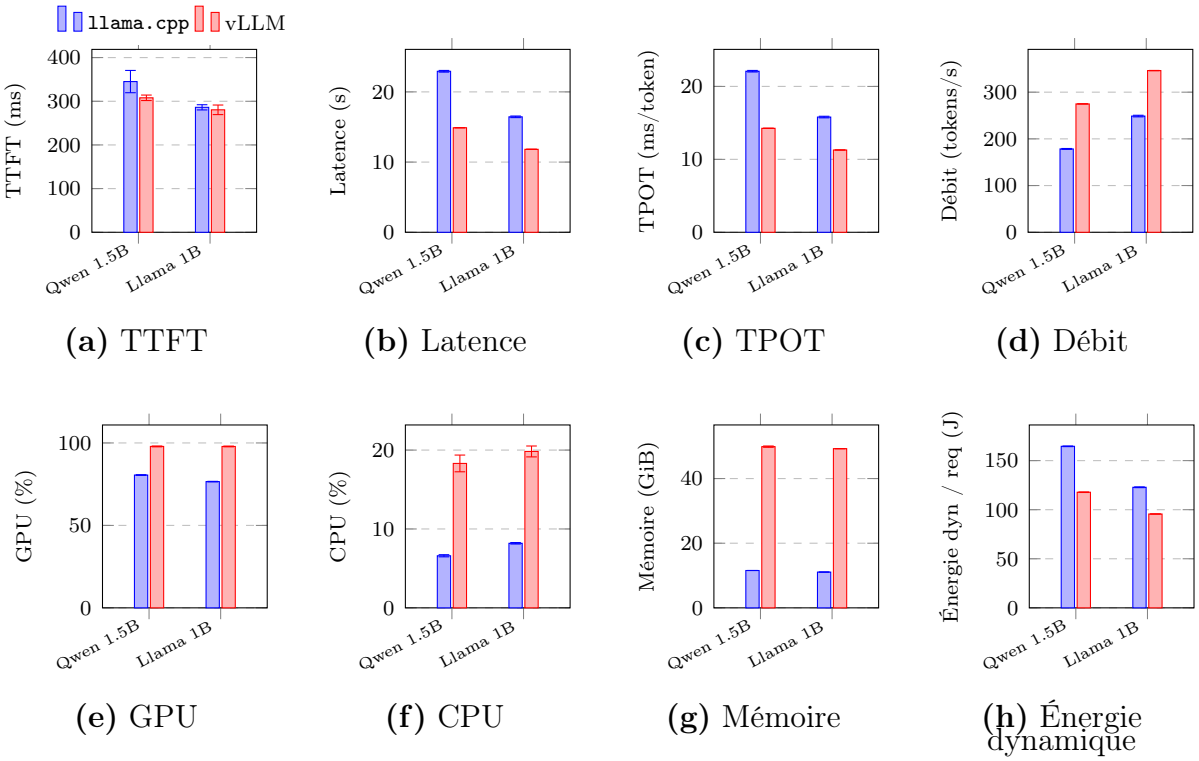
(e) GPU **(f)** CPU **(g)** Mémoire **(h)** Énergie dynamique

FIGURE 4.8 – Généralisation des résultats à une augmentation de la taille du modèle pour $P128/O1024$ à $C = 4$.

Généralisation à une taille de modèle supérieure. L'augmentation de la taille au sein de Qwen2.5 dégrade les performances des deux moteurs sans modifier leur classement global. Lorsque le modèle passe de 1,5 à 3,09 milliards de paramètres, le TTFT augmente d'environ 37 % avec `llama.cpp`, contre 16 % avec vLLM (Figure 4.8(a)). L'effet est plus marqué pendant le *decode* : la latence et le TPOT augmentent tous deux d'environ 48 % avec `llama.cpp`, contre respectivement 78 % et 79 % avec vLLM (Figures 4.8(b) et (c)). Le débit diminue en conséquence de 32 % avec `llama.cpp` et de 44 % avec vLLM (Figure 4.8(d)). Malgré une dégradation relative plus forte de vLLM pendant la génération, celui-ci conserve avec le modèle 3B un TTFT, une latence et un TPOT plus faibles ainsi qu'un débit supérieur.

Du côté des ressources, l'utilisation GPU augmente avec la taille du modèle pour `llama.cpp`, d'environ 81 à 86 %, tandis qu'elle reste proche de 98 % avec vLLM (Figure 4.8(e)). L'utilisation CPU diminue légèrement pour les deux moteurs (Figure 4.8(f)). Cette diminution ne permet toutefois pas de conclure à une réduction du travail côté hôte : elle peut notamment refléter le fait que l'exécution GPU devient plus longue, de sorte que l'activité CPU représente une fraction plus faible de la fenêtre de mesure. La mémoire unifiée totale varie peu (Figure 4.8(g)), notamment en raison des allocations fixes des moteurs et de la préallocation de vLLM déjà décrite en Q1. L'effet de la taille apparaît en revanche clairement sur l'énergie dynamique, qui augmente d'environ 87 % pour les deux moteurs (Figure 4.8(h)).

L'augmentation du nombre de paramètres conserve donc le compromis observé précédemment : vLLM reste plus performant et conserve une énergie dynamique par requête plus faible, tandis que `llama.cpp` conserve une occupation CPU et mémoire plus faible.



(a) TTFT **(b)** Latence **(c)** TPOT **(d)** Débit

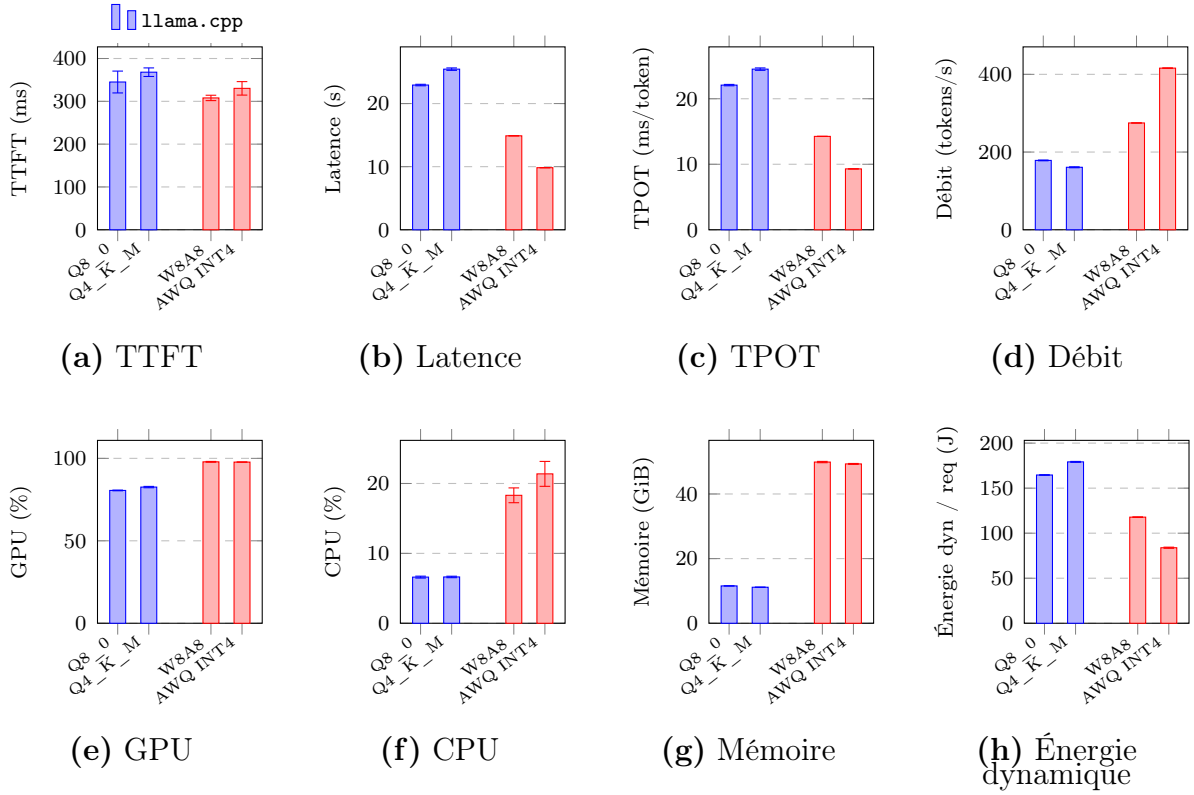
(e) GPU **(f)** CPU **(g)** Mémoire **(h)** Énergie dynamique

FIGURE 4.9 – Généralisation des résultats à une autre famille de modèles pour $P128/O1024$ à $C = 4$.

Généralisation à une autre famille de modèles. Le passage de Qwen2.5-1.5B à Llama 3.2-1B s'accompagne d'une amélioration des performances des deux moteurs, mais modifie l'amplitude de leur écart. Le TTFT diminue d'environ 17 % avec `llama.cpp` et de 9 % avec vLLM, jusqu'à devenir presque identique entre les deux moteurs, autour de 0,28 s (Figure 4.9(a)). La latence et le TPOT diminuent également avec Llama 3.2-1B (Figures 4.9(b) et (c)), mais vLLM conserve son avantage, avec un TPOT d'environ 11,3 ms/*token* contre 15,8 ms/*token* avec `llama.cpp`. Le débit augmente parallèlement d'environ 39 % avec `llama.cpp` et de 26 % avec vLLM (Figure 4.9(d)), tout en restant supérieur avec vLLM.

Les différences de ressources observées précédemment restent également visibles. vLLM

maintient une utilisation GPU proche de 98 %, tandis que celle de `llama.cpp` reste plus faible (Figure 4.9(e)). L'utilisation CPU de vLLM demeure également supérieure (Figure 4.9(f)). La mémoire unifiée totale reste également nettement plus faible avec `llama.cpp` (Figure 4.9(g)). Le changement de modèle réduit enfin l'énergie dynamique d'environ 25 % avec `llama.cpp` et de 19 % avec vLLM (Figure 4.9(h)). Ces différences ne peuvent cependant pas être attribuées exclusivement à l'architecture, puisque Llama 3.2-1B comporte également moins de paramètres. Elles montrent néanmoins que le changement de famille modifie sensiblement les valeurs absolues, en particulier le TTFT, sans remettre en cause les tendances principales observées sur la latence, le TPOT, le débit et les ressources.



(a) TTFT **(b)** Latence **(c)** TPOT **(d)** Débit

(e) GPU **(f)** CPU **(g)** Mémoire **(h)** Énergie dynamique

FIGURE 4.10 – Influence de la quantification sur les performances, les ressources et l'énergie pour Qwen2.5-1.5B-Instruct à $P128/O1024$ et $C = 4$.

Généralisation à une quantification de plus faible précision. La réduction de précision produit en revanche des effets différents selon le moteur. Avec `llama.cpp`, le passage de Q8_0 à Q4_K_M ne conduit pas à une accélération : le TTFT augmente d'environ 7 % (Figure 4.10(a)), tandis que la latence et le TPOT augmentent d'environ 11 % (Figures 4.10(b) et (c)). Le débit diminue ainsi d'environ 10 % (Figure 4.10(d)). La réduction de la représentation des poids ne se traduit donc pas automatiquement par une réduction du temps d'exécution. Ce comportement est cohérent avec les observations de Bouzouad et al., qui montrent que, pour les K-types de `llama.cpp`, une précision plus faible peut aussi bien améliorer que dégrader la latence selon le modèle, le matériel et le coût du chemin d'exécution [6].

Avec vLLM, le passage de W8A8 à AWQ INT4 augmente lui aussi légèrement le TTFT, d'environ 7 %, mais améliore fortement le *decode* : la latence diminue de 34 % et le TPOT

de 35 %, tandis que le débit augmente de 51 % (Figures 4.10(a)–(d)). Le gain associé à la quantification apparaît donc principalement pendant la génération. Cette différence avec `llama.cpp` ne signifie pas qu'AWQ est intrinsèquement supérieur à Q4_K_M, puisque les deux formats et leurs chemins d'exécution diffèrent ; elle montre plutôt que le bénéfice d'une réduction de précision dépend de la capacité du moteur à l'exploiter efficacement sur le matériel cible.

Les ressources renforcent cette interprétation. Avec `llama.cpp`, Q4_K_M augmente légèrement l'utilisation GPU tout en étant plus lent (Figure 4.10(e)), tandis qu'avec vLLM l'occupation GPU reste pratiquement inchangée autour de 98 % malgré la forte augmentation du débit. L'utilisation CPU varie peu entre Q8_0 et Q4_K_M, mais augmente avec AWQ INT4 (Figure 4.10(f)). La mémoire unifiée totale ne diminue que faiblement dans les deux cas (Figure 4.10(g)), car la métrique reste dominée par les allocations globales des moteurs plutôt que par les seuls poids. Enfin, l'énergie dynamique évolue dans le même sens que la durée d'exécution : Q4_K_M l'augmente d'environ 9 % avec `llama.cpp`, alors qu'AWQ INT4 la réduit d'environ 29 % avec vLLM (Figure 4.10(h)). Une précision plus faible ne garantit donc ni de meilleures performances ni une énergie inférieure ; le format de quantification et son chemin d'exécution sur le matériel cible restent déterminants.

Ces résultats répondent à Q4 : les tendances principales observées avec le modèle de référence se maintiennent lorsque sa taille augmente ou lorsqu'une autre famille de modèles est utilisée. vLLM conserve globalement une latence et un TPOT plus faibles, un débit supérieur et une énergie dynamique par requête inférieure, tandis que `llama.cpp` reste plus léger en CPU et en mémoire. L'amplitude de ces écarts dépend néanmoins du modèle, notamment pour le TTFT, qui devient presque équivalent entre les moteurs avec Llama 3.2-1B. La quantification constitue le cas où la généralisation est la moins directe : la réduction de précision dégrade ici les performances de `llama.cpp` avec Q4_K_M, alors qu'elle améliore fortement le *decode* et réduit l'énergie dynamique de vLLM avec AWQ INT4. Les effets d'une quantification doivent donc être interprétés conjointement avec le moteur et le chemin d'exécution réellement utilisé.

4.6 Généralisation à une autre plateforme *Edge*

Les campagnes précédentes ont permis d'établir les principaux comportements de `llama.cpp` et de vLLM sur la Jetson AGX Orin. Cette dernière campagne cherche à déterminer dans quelle mesure ces observations dépendent de la plateforme matérielle considérée. Une sélection représentative des expériences précédentes est donc reproduite sur la Jetson Orin Nano, dont les capacités de calcul et la mémoire disponible sont plus limitées. L'objectif n'est pas de réaliser une nouvelle caractérisation exhaustive, mais de vérifier si les effets associés à la charge, à la concurrence et à la quantification, ainsi que le compromis entre les deux moteurs, se maintiennent lorsque les ressources matérielles changent.

4.6.1 Configuration de l'expérience

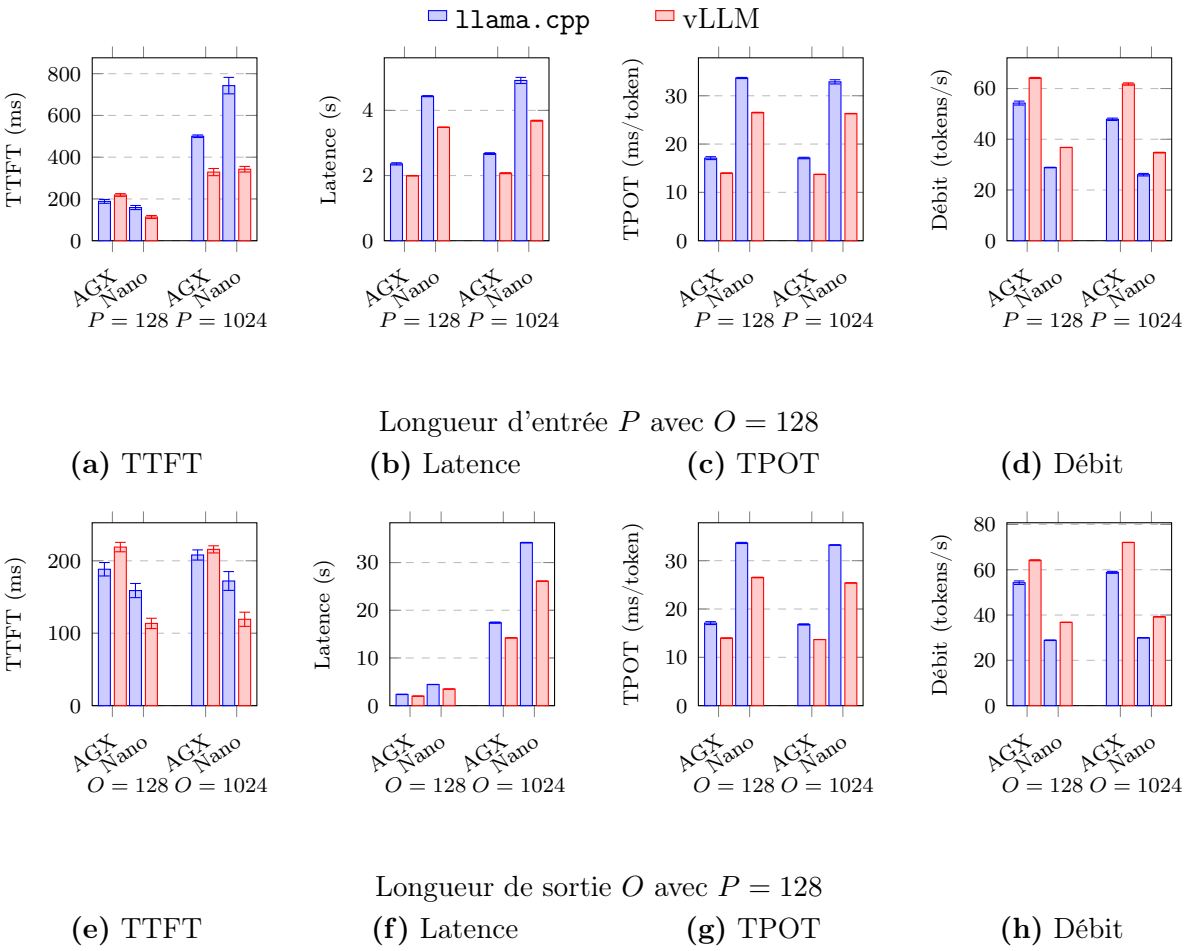
Les deux plateformes conservent leur configuration de fonctionnement définie dans le chapitre précédent. Le modèle de référence reste Qwen2.5-1.5B-Instruct, utilisé en Q8_0 avec `llama.cpp` et en W8A8 avec vLLM. Les deux moteurs conservent également une capacité maximale de quatre séquences actives. La seule adaptation nécessaire à la capacité

mémoire du Nano concerne vLLM, pour lequel `gpu-memory-utilization` est fixé à 0,40, contre 0,70 sur l'AGX.

Trois expériences sont retenues. L'effet de la charge est d'abord réévalué à $C = 1$ à partir des profils $P128/O128$, $P1024/O128$ et $P128/O1024$, ce qui permet de retrouver séparément l'allongement du *prefill* et du *decode* étudiés en Q1. La concurrence est ensuite examinée sur $P128/O1024$ pour $C = 1$, 4 et 8, afin de couvrir à la fois l'augmentation du parallélisme actif et le régime de mise en attente identifié en Q2. Enfin, l'effet de la quantification est réévalué sur $P128/O1024$ à $C = 4$, en comparant Q8_0 à Q4_K_M pour `llama.cpp` et W8A8 à AWQ INT4 pour vLLM.

4.6.2 Résultats et analyse

Les Figures 4.11 et 4.12 comparent d'abord la généralisation de l'effet de la charge, respectivement sur les performances côté client et sur les ressources mobilisées. Les Figures 4.13 et 4.14 présentent ensuite les résultats liés à la concurrence, tandis que la Figure 4.15 regroupe ceux de la quantification. Dans cette dernière campagne, l'analyse porte principalement sur la conservation des tendances et du classement entre moteurs plutôt que sur les seules différences absolues de performance entre les deux plateformes.



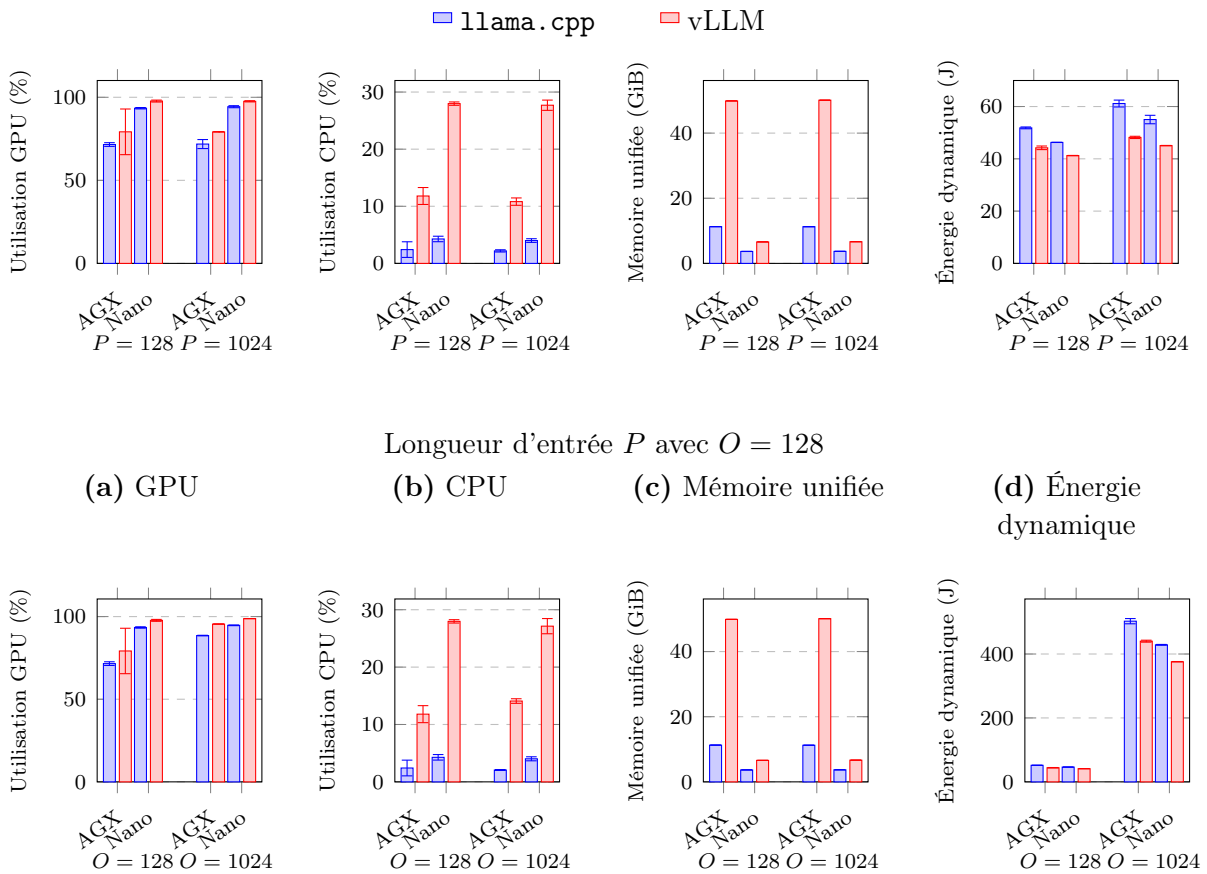
Longueur d'entrée P avec $O = 128$

(a) TTFT **(b)** Latence **(c)** TPOT **(d)** Débit

Longueur de sortie O avec $P = 128$

(e) TTFT **(f)** Latence **(g)** TPOT **(h)** Débit

FIGURE 4.11 – Généralisation de l'effet de la charge sur les performances côté client entre l'AGX Orin et l'Orin Nano à $C = 1$.



Longueur d'entrée P avec $O = 128$

(a) GPU **(b)** CPU **(c)** Mémoire unifiée **(d)** Énergie dynamique

Longueur de sortie O avec $P = 128$

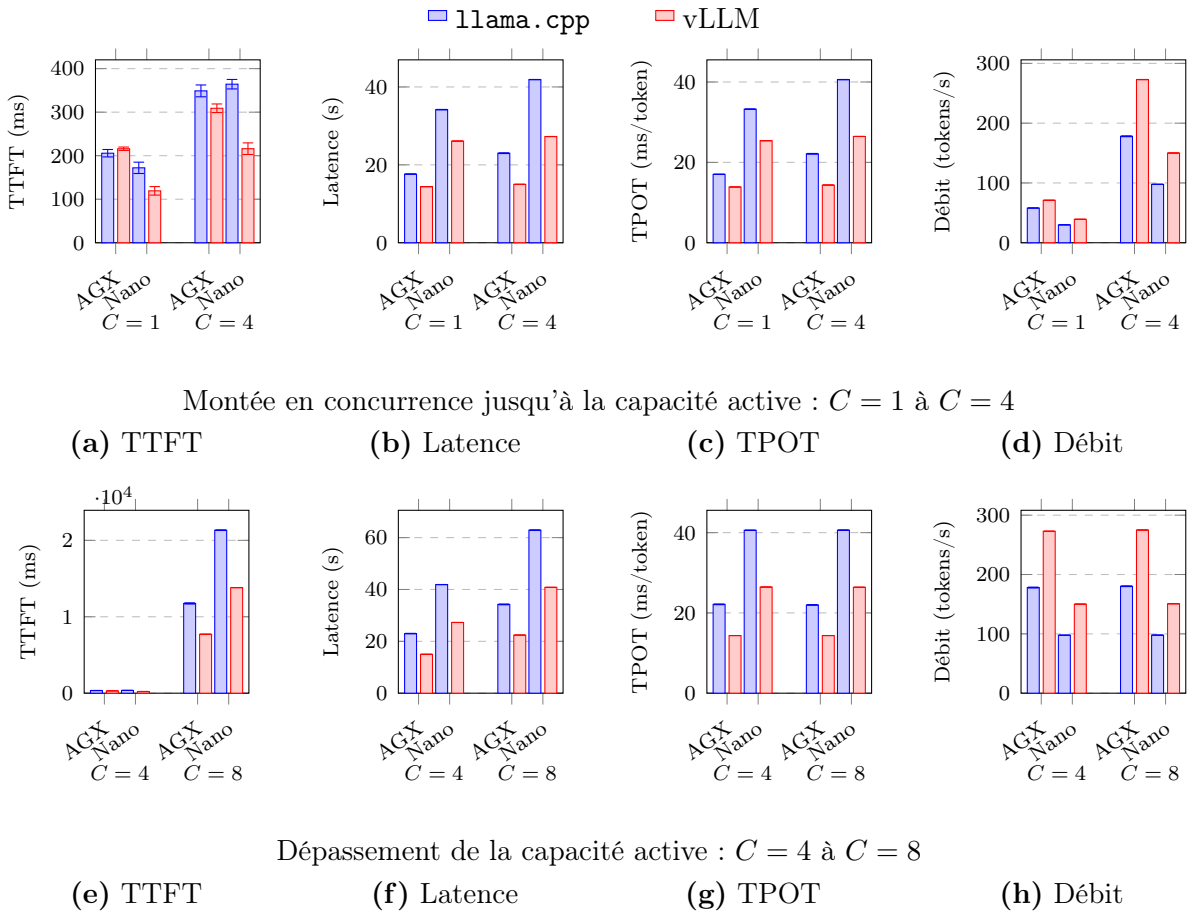
(e) GPU **(f)** CPU **(g)** Mémoire unifiée **(h)** Énergie dynamique

FIGURE 4.12 – Généralisation de l'effet de la charge sur les ressources mobilisées et l'énergie entre l'AGX Orin et l'Orin Nano à $C = 1$.

Généralisation de l'effet de la charge. La Figure 4.11 montre que l'Orin Nano reproduit globalement les tendances observées sur l'AGX Orin, avec des performances absolues généralement plus faibles. L'allongement du *prompt* affecte principalement le TTFT, tandis que l'augmentation de la longueur de sortie accroît surtout la latence, avec un TPOT qui reste relativement stable. Sur le Nano comme sur l'AGX, vLLM conserve ainsi son avantage sur `llama.cpp` en termes de latence, de TPOT et de débit, avec un écart relatif moyen d'environ 20 % sur l'AGX contre 25 % sur le Nano. Toutefois, le comportement du TTFT diffère entre les deux plateformes. Pour un *prompt* court, `llama.cpp` présente un léger avantage sur l'AGX, alors que le classement s'inverse sur le Nano, où vLLM atteint environ 114–119 ms contre 159–172 ms pour `llama.cpp` (Figures 4.11(a) et (e)). Le TTFT de vLLM sur le Nano est même inférieur à celui observé sur l'AGX, qui se situe autour de 216–219 ms pour les mêmes profils. Cette inversion a été reproduite dans un test complémentaire, mais son origine précise n'a pas pu être isolée. Pour un *prompt* long, vLLM conserve également le TTFT le plus faible sur les deux plateformes.

Du côté des ressources, la Figure 4.12 montre que les principaux écarts entre les deux moteurs se maintiennent sur l'Orin Nano : vLLM conserve une utilisation GPU élevée, sollicite davantage le CPU et la mémoire, et présente une énergie dynamique par requête

plus faible. Les niveaux absolus d'utilisation des ressources ne sont toutefois pas directement comparables entre les deux plateformes, qui disposent de capacités matérielles différentes.



Montée en concurrence jusqu'à la capacité active : $C = 1$ à $C = 4$

(a) TTFT **(b)** Latence **(c)** TPOT **(d)** Débit

Dépassement de la capacité active : $C = 4$ à $C = 8$

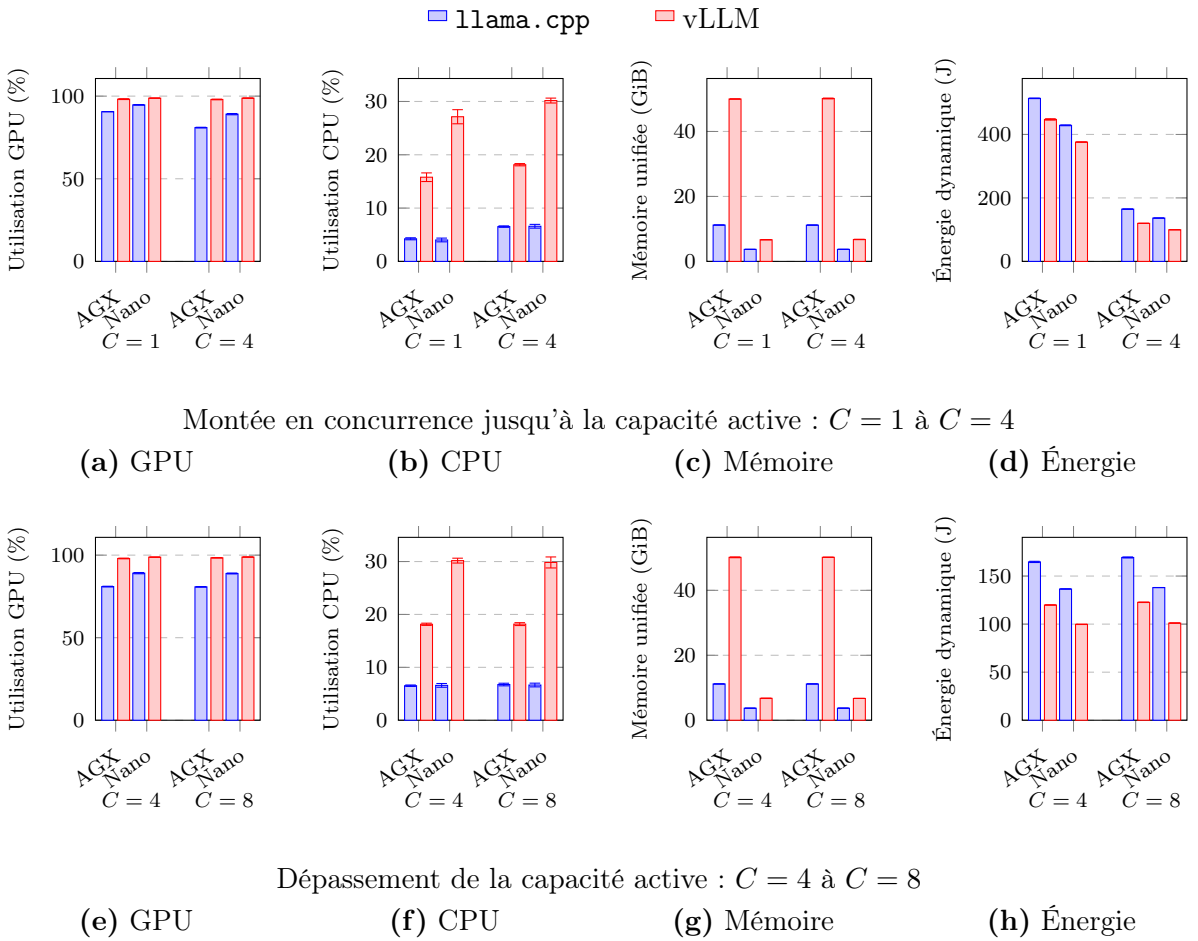
(e) TTFT **(f)** Latence **(g)** TPOT **(h)** Débit

FIGURE 4.13 – Généralisation de l'effet de la concurrence sur les performances côté client entre l'AGX Orin et l'Orin Nano pour $P128/O1024$.

Généralisation de l'effet de la concurrence. La Figure 4.13 montre que l'Orin Nano reproduit globalement les tendances observées sur l'AGX Orin jusqu'à $C = 4$. La latence et le TPOT augmentent avec la concurrence, tandis que le débit progresse, avec une meilleure efficacité du parallélisme pour vLLM. L'avantage relatif de vLLM est presque identique sur les deux plateformes à $C = 4$: sa latence et son TPOT sont inférieurs d'environ 35 % à ceux de `llama.cpp`, tandis que son débit est supérieur d'environ 53 %. Toutefois, comme pour l'effet de la charge, le TTFT présente un comportement plus dépendant de la plateforme : à $C = 4$, vLLM présente un TTFT inférieur d'environ 11 % à celui de `llama.cpp` sur l'AGX, contre environ 41 % sur le Nano.

Au-delà de $C = 4$, le même changement de régime apparaît sur les deux plateformes : le débit et le TPOT se stabilisent, tandis que le TTFT et la latence augmentent sous l'effet de la mise en attente. À $C = 8$, l'avantage relatif de vLLM est presque identique sur les deux plateformes, avec un TTFT, une latence et un TPOT inférieurs d'environ 35 % et un débit supérieur d'environ 53–54 %.

La Figure 4.14 retrouve également les mêmes tendances du côté des ressources, notamment l'utilisation GPU plus élevée de vLLM, cohérente avec l'activité GPU plus continue mise en évidence par l'analyse Nsight en Q2.



Montée en concurrence jusqu'à la capacité active : $C = 1$ à $C = 4$

(a) GPU **(b)** CPU **(c)** Mémoire **(d)** Énergie

Dépassement de la capacité active : $C = 4$ à $C = 8$

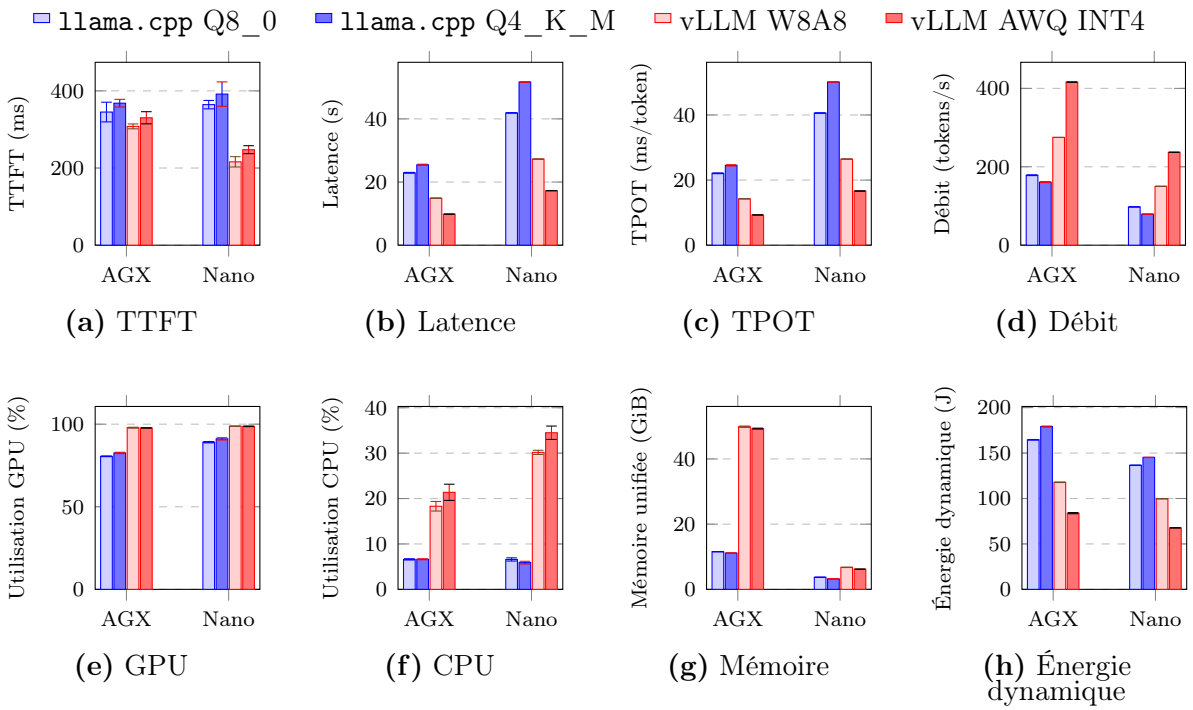
(e) GPU **(f)** CPU **(g)** Mémoire **(h)** Énergie

FIGURE 4.14 – Généralisation de l'effet de la concurrence sur les ressources mobilisées et l'énergie entre l'AGX Orin et l'Orin Nano pour $P128/O1024$.

Généralisation de l'effet de la quantification. La Figure 4.15 montre que l'effet de la quantification reste fortement dépendant du moteur sur l'Orin Nano. Dans les deux moteurs, une quantification plus agressive n'améliore pas le TTFT, qui augmente légèrement. En revanche, son effet sur le *decode* reste opposé : avec `llama.cpp`, le passage de Q8_0 à Q4_K_M dégrade le TPOT d'environ 11 % sur l'AGX et 23 % sur le Nano, avec une baisse du débit d'environ 10 % et 19 %. À l'inverse, avec vLLM, AWQ INT4 réduit le TPOT d'environ 35 % sur l'AGX et 37 % sur le Nano, tandis que le débit augmente respectivement de 51 % et 58 %. La direction de l'effet observé en Q4 est donc conservée sur les deux plateformes.

Du côté des ressources, les mêmes tendances sont retrouvées : l'énergie dynamique augmente avec Q4_K_M pour `llama.cpp`, alors qu'elle diminue nettement avec AWQ INT4 pour vLLM. Ces résultats confirment ainsi qu'une réduction de précision n'entraîne pas systématiquement une accélération de l'inférence et que son effet dépend du format de quantification et du chemin d'exécution exploité par le moteur.

Ces résultats répondent à Q5 : le passage à une plateforme *Edge* plus contrainte modifie les performances absolues, mais les principaux mécanismes identifiés sur l'AGX Orin se maintiennent globalement. Les effets de la charge, les deux régimes de concurrence et la dépendance de la quantification au moteur se retrouvent sur l'Orin Nano, et vLLM y conserve son avantage global en latence, TPOT, débit et énergie dynamique. Le TTFT à faible charge constitue toutefois une exception, puisque le léger avantage de `llama.cpp`



(a) TTFT **(b)** Latence **(c)** TPOT **(d)** Débit

(e) GPU **(f)** CPU **(g)** Mémoire **(h)** Énergie dynamique

FIGURE 4.15 – Généralisation de l'effet de la quantification entre l'AGX Orin et l'Orin Nano pour Qwen2.5-1.5B-Instruct, $P128/O1024$ et $C = 4$.

observé sur l'AGX s'inverse sur le Nano. Les conclusions principales se généralisent donc à l'Orin Nano, mais l'amplitude des écarts et certains comportements restent dépendants de la plateforme.

4.7 Synthèse du chapitre

Les cinq questions étudiées dans ce chapitre ont permis de comparer `llama.cpp` et vLLM selon la charge de travail, la concurrence, le partage des ressources, le modèle et la quantification, puis la plateforme. Les résultats obtenus permettent maintenant de regrouper les principaux comportements observés et d'identifier le compromis entre les deux moteurs dans un contexte *Edge* multi-utilisateur.

4.7.1 Synthèse des résultats

Le Tableau 4.1 regroupe les principaux résultats obtenus pour les cinq questions étudiées dans ce chapitre.

Pris ensemble, ces résultats montrent que vLLM conserve l'avantage le plus régulier sur la latence, le TPOT, le débit et l'exploitation de la concurrence. `llama.cpp` mobilise en revanche moins de CPU et de mémoire et se dégrade proportionnellement moins lorsque ces ressources sont partagées. Le compromis entre les deux moteurs dépend donc principalement des performances recherchées et des ressources disponibles sur la plateforme.

TABLE 4.1 – Synthèse des principaux résultats de Q1 à Q5.

Question	Résultats clés
Q1 : Charge de travail	Pour un *prompt* court, le TTFT de `llama.cpp` est inférieur d'environ 14 %, puis le classement s'inverse à $P = 1024$, où celui de vLLM est inférieur d'environ 34 %. Sur le *decode*, vLLM réduit le TPOT de 18 à 20 % et augmente le débit de 18 à 29 %.
Q2 : Concurrence	À $C = 4$, l'efficacité du parallélisme atteint environ 77 % avec `llama.cpp` contre 96 % avec vLLM. Au-delà, le débit plafonne alors que le TTFT et la latence augmentent avec la mise en attente.
Q3 : Partage des ressources	À 75 % de contention, la latence augmente de 21 % avec `llama.cpp` contre 41 % avec vLLM, et le TPOT de 22 % contre 42 %. `llama.cpp` mobilise également moins de CPU et de mémoire.
Q4 : Modèle et quantification	**Taille et famille :** vLLM conserve globalement son avantage en latence, TPOT et débit, tandis que le TTFT varie davantage. **Quantification :** Q4_K_M augmente le TPOT de `llama.cpp` de 11 %, alors qu'AWQ INT4 réduit celui de vLLM de 35 % et augmente son débit de 51 %.
Q5 : Plateforme	À $C = 4$, vLLM conserve sur les deux plateformes une latence et un TPOT environ 35 % plus faibles et un débit environ 53 % plus élevé. Le TTFT à faible charge constitue l'exception, puisque le classement observé sur l'AGX s'inverse sur le Nano.

4.7.2 Choix du moteur selon la charge et les ressources disponibles

Lorsque les performances du service LLM constituent la contrainte principale, les résultats obtenus favorisent vLLM. Son avantage est particulièrement régulier pendant le *decode* et lorsque plusieurs requêtes sont exécutées en parallèle. Ce choix doit néanmoins tenir compte du TTFT lorsque cette métrique est prioritaire, puisque les résultats montrent que son classement peut changer selon la charge et la plateforme.

Le compromis devient différent lorsque le LLM partage la plateforme avec d'autres applications. Dans cette situation, il ne suffit plus de considérer uniquement les performances du moteur, puisque les ressources qu'il mobilise réduisent celles disponibles pour les autres charges. Les résultats de Q3 montrent alors que `llama.cpp` utilise moins de CPU et de mémoire et que ses performances se dégradent proportionnellement moins lorsque la contention augmente. Il peut donc devenir intéressant lorsque la disponibilité de ces ressources constitue une contrainte importante du déploiement.

La caractérisation réalisée dans ce chapitre conduit ainsi à une piste d'optimisation fondée sur le choix du moteur selon la charge et les ressources disponibles. vLLM peut être privilégié lorsque les performances du service LLM sont prioritaires, tandis que `llama.cpp` peut être considéré lorsque la plateforme doit conserver davantage de CPU et de mémoire pour d'autres applications. Les résultats de Q4 et Q5 montrent toutefois que ce choix doit être vérifié pour le modèle et la plateforme considérés, puisque certains écarts peuvent varier.

Cette sélection du moteur n'est pas mise en œuvre dans ce travail. Les résultats obtenus fournissent cependant les éléments nécessaires pour définir, dans un travail futur, une stratégie de choix tenant compte à la fois de la charge du service LLM et des ressources disponibles sur la plateforme.

4.8 Conclusion

Ce chapitre a permis de caractériser le comportement de `llama.cpp` et de vLLM dans différentes conditions représentatives d'un déploiement *Edge* multi-utilisateur. Les résultats montrent que vLLM conserve globalement l'avantage sur la latence, le TPOT et le débit, notamment lorsque la charge ou la concurrence augmente. À l'inverse, `llama.cpp` mobilise moins de CPU et de mémoire et se dégrade proportionnellement moins lorsque ces ressources sont partagées.

Les campagnes menées sur différents modèles, quantifications et plateformes montrent également que ces tendances restent globalement valables, même si certains comportements, en particulier le TTFT et l'effet de la quantification, dépendent davantage de la configuration considérée. Le choix du moteur ne peut donc pas être établi indépendamment de la charge de travail et des ressources disponibles sur la plateforme. Cette caractérisation fournit ainsi une base pour envisager une sélection du moteur adaptée aux conditions d'exécution dans un environnement *Edge* multi-utilisateur.

Conclusion générale

Ce mémoire présente le travail réalisé à ENSTA campus Brest, au sein du Lab-STICC, autour de la caractérisation de l'inférence multi-utilisateur de modèles de langage sur des plateformes *Edge*. L'objectif principal était d'étudier le comportement de l'inférence dans ce contexte en considérant conjointement les performances côté utilisateur et les ressources système mobilisées, puis d'identifier les principaux facteurs susceptibles de les influencer.

Une étude bibliographique a d'abord permis de situer ce travail par rapport aux systèmes d'inférence et aux études de caractérisation existantes. Elle a montré que les travaux multi-utilisateur concernent principalement des infrastructures serveur, tandis que les études *Edge* se concentrent davantage sur la charge, le modèle, la quantification ou la plateforme [4, 5, 24, 7, 3, 1]. À partir de cette analyse, `llama.cpp` et vLLM ont été retenus comme deux moteurs reposant sur des choix de conception différents. Plusieurs axes de caractérisation ont ensuite été définis : la charge de travail, la concurrence, le partage des ressources, le modèle, la quantification et la plateforme.

Une méthodologie expérimentale commune a ensuite été mise en place afin de comparer les deux moteurs dans des conditions aussi similaires que possible. Les campagnes ont été organisées progressivement autour de cinq questions de recherche, depuis l'influence de la charge jusqu'à la généralisation des observations à d'autres modèles, quantifications et à une seconde plateforme *Edge*.

Les résultats montrent que vLLM offre les meilleures performances dans la majorité des configurations étudiées, notamment en termes de latence, de TPOT et de débit, avec un avantage plus marqué lorsque la charge ou la concurrence augmente. Cet avantage s'accompagne toutefois d'une utilisation plus importante du CPU et surtout de la mémoire. À l'inverse, `llama.cpp` présente une empreinte système plus légère et se montre proportionnellement moins sensible à la contention CPU et mémoire, sans devenir globalement plus performant dans les scénarios considérés. L'étude montre également que la réduction de précision ne conduit pas systématiquement à une accélération, son effet dépendant du moteur et du chemin d'exécution associé. Enfin, les expériences sur la Jetson Orin Nano confirment plusieurs tendances observées sur l'AGX Orin, même si certaines métriques, notamment le TTFT à faible charge, restent dépendantes de la plateforme. Dans l'ensemble, ces résultats mettent ainsi en évidence un compromis entre performances et ressources mobilisées plutôt qu'une supériorité absolue d'un moteur dans tous les contextes.

Difficultés rencontrées

La principale difficulté de cette étude tient à l'étendue du problème de caractérisation. Les performances dépendent simultanément de nombreux facteurs liés au moteur, au modèle, à la charge et à la plateforme, ce qui a nécessité de délimiter un périmètre expéri-

mental suffisamment représentatif tout en conservant un nombre d'expériences compatible avec la durée du stage.

La comparaison de `llama.cpp` et de vLLM a également demandé la définition d'un cadre commun malgré leurs différences de fonctionnement et de configuration.

Enfin, l'évolution rapide de l'écosystème des LLM a constitué une difficulté supplémentaire. Afin de préserver la cohérence et la reproductibilité des campagnes, les versions des moteurs, les environnements logiciels et les principales configurations ont dû être figés pendant l'étude.

Limites de l'étude

Malgré l'étendue des campagnes réalisées, plusieurs limites doivent être soulignées :

— `llama.cpp` et vLLM proposent un grand nombre de paramètres de configuration susceptibles d'influencer leurs performances. Les résultats correspondent ainsi aux configurations étudiées et non nécessairement aux performances maximales que chaque moteur pourrait atteindre après une optimisation spécifique.

— L'étude se limite à deux moteurs d'inférence. D'autres moteurs reposant sur des mécanismes différents pourraient présenter d'autres compromis entre performances et ressources système.

— Les expérimentations ont été réalisées sur deux plateformes NVIDIA Jetson. Cette sélection permet d'étudier deux niveaux de ressources différents, mais ne couvre pas l'ensemble des architectures *Edge* existantes.

— Les modèles ont principalement été exécutés sur GPU. Les scénarios reposant essentiellement sur le CPU, notamment sur des plateformes telles que Raspberry Pi, ainsi que d'autres répartitions possibles de l'exécution entre CPU et GPU, n'ont pas été étudiés.

Perspectives

Ces limites ouvrent plusieurs perspectives :

1. **Exploration des configurations des moteurs** : étudier davantage les paramètres de `llama.cpp` et de vLLM afin d'identifier des configurations mieux adaptées aux différentes charges et plateformes ;
2. **Extension à d'autres moteurs et plateformes** : élargir la caractérisation à d'autres moteurs, modèles et architectures *Edge*, notamment à des plateformes principalement CPU ;
3. **Extension vers un scénario multi-modèle** : étudier l'exécution simultanée de plusieurs modèles sur une même plateforme et leur partage des ressources CPU, GPU et mémoire ;
4. **Aide au choix du moteur et de sa configuration** : exploiter une caractérisation plus large afin de développer un outil capable de recommander un moteur et une configuration en fonction de la plateforme, du modèle, de la charge attendue et des contraintes de performances ou de ressources.

Bibliographie

[1] Maximilian Abstreiter, Sasu Tarkoma et Roberto Morabito. « Sometimes Painful but Promising : Feasibility and Trade-Offs of On-Device Language Model Inference ». In : *ACM Trans. Embed. Comput. Syst.* 25.2 (mars 2026). ISSN : 1539-9087. DOI : 10.1145/3788870. URL : https://doi.org/10.1145/3788870.

[2] Amey Agrawal et al. « Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve ». In : *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA : USENIX Association, juill. 2024, p. 117-134. ISBN : 978-1-939133-40-3. URL : https://www.usenix.org/conference/osdi24/presentation/agrawal.

[3] Mayank Arya et Yogesh Simmhan. « Understanding the Performance and Power of LLM Inferencing on Edge Accelerators ». In : *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2025, p. 1108-1111. DOI : 10.1109/IPDPSW66978.2025.00173.

[4] Betül Ay et Yunus Emre Demirdağ. « Benchmarking Ollama and vLLM for Concurrent LLM Serving : A Multi-Scenario Evaluation of Performance and Scalability ». In : *Applied Sciences* 16.11 (2026), p. 5435. DOI : 10.3390/app16115435.

[5] Austin Bomhold et al. « InferenceProfiler : Enhancing the Observability of Performance, Cost, and Energy of LLM Inferencing in the Cloud ». In : *Proceedings of the 20th ACM International Conference on Distributed and Event-Based Systems*. DEBS '26. Universidade de Lisboa, Lisbon, Portugal : Association for Computing Machinery, 2026, p. 17-22. ISBN : 9798400726934. DOI : 10.1145/3809481.3816482. URL : https://doi.org/10.1145/3809481.3816482.

[6] Meriem Bouzouad, Yuan-Hao Chang et Jalil Boukhobza. *APreQEL : Adaptive Mixed Precision Quantization for Edge LLMs*. Mars 2026. arXiv : 2603.23575 [cs.LG]. URL : https://arxiv.org/abs/2603.23575.

[7] Nobel Dhar et al. « An Empirical Analysis and Resource Footprint Study of Deploying Large Language Models on Edge Devices ». In : *Proceedings of the 2024 ACM Southeast Conference*. ACMSE '24. Marietta, GA, USA : Association for Computing Machinery, 2024, p. 69-76. ISBN : 9798400702372. DOI : 10.1145/3603287.3651205. URL : https://doi.org/10.1145/3603287.3651205.

[8] Yichao Fu et al. « Efficient LLM Scheduling by Learning to Rank ». In : *Advances in Neural Information Processing Systems*. T. 37. 2024. DOI : 10.52202/079017-1882.

[9] ggml-org. *llama.cpp : LLM inference in C/C++*. URL : https://github.com/ggml-org/llama.cpp (visité le 25/08/2026).

[10] Henrikas GIEDRA et al. « Benchmarking Large Language Model Inference on Limited-Resource Edge Systems ». In : *Electronics* 15.11 (2026). ISSN : 2079-9292. DOI : 10.3390/electronics15112451. URL : https://www.mdpi.com/2079-9292/15/11/2451.

[11] Erik Johannes HUSOM et al. « Sustainable LLM Inference for Edge AI : Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency ». In : *ACM Trans. Internet Things* 6.4 (nov. 2025). DOI : 10.1145/3767742. URL : https://doi.org/10.1145/3767742.

[12] Erik Johannes HUSOM et al. « Sustainable LLM Inference for Edge AI : Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency ». In : *ACM Transactions on Internet of Things* 6.4 (nov. 2025). DOI : 10.1145/3767742. URL : https://doi.org/10.1145/3767742.

[13] Woosuk KWON et al. « Efficient Memory Management for Large Language Model Serving with PagedAttention ». In : *Proceedings of the 29th Symposium on Operating Systems Principles*. SOSP '23. Koblenz, Germany : Association for Computing Machinery, 2023, p. 611-626. ISBN : 9798400702297. DOI : 10.1145/3600006.3613165. URL : https://doi.org/10.1145/3600006.3613165.

[14] Stefanos LASKARIDIS et al. « MELTing Point : Mobile Evaluation of Language Transformers ». In : *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. ACM MobiCom '24. Washington D.C., DC, USA : Association for Computing Machinery, 2024, p. 890-907. ISBN : 9798400704895. DOI : 10.1145/3636534.3690668. URL : https://doi.org/10.1145/3636534.3690668.

[15] Malgorzata LAZUKA, Andreea ANGHEL et Thomas PARNELL. « LLM-Pilot : Characterize and Optimize Performance of your LLM Inference Services ». In : *SC24 : International Conference for High Performance Computing, Networking, Storage and Analysis*. 2024, p. 1-18. DOI : 10.1109/SC41406.2024.00022.

[16] Baolin LI et al. « LLM Inference Serving : Survey of Recent Advances and Opportunities ». In : *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. 2024, p. 1-8. DOI : 10.1109/HPEC62836.2024.10938426.

[17] Baolin LI et al. « LLM Inference Serving : Survey of Recent Advances and Opportunities ». In : *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. 2024, p. 1-8. DOI : 10.1109/HPEC62836.2024.10938426.

[18] Ruihao LI et al. « Oneiros : KV Cache Optimization through Parameter Remapping for Multi-tenant LLM Serving ». In : *Proceedings of the 2025 ACM Symposium on Cloud Computing*. SoCC '25. Association for Computing Machinery, 2026, p. 88-101. ISBN : 9798400722769. DOI : 10.1145/3772052.3772215. URL : https://doi.org/10.1145/3772052.3772215.

[19] Ji LIN et al. « AWQ : Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration ». In : *Proceedings of Machine Learning and Systems* 6 (2024), p. 87-100. URL : https://proceedings.mlsys.org/paper_files/paper/2024/hash/42a452cbafa9dd64e9ba4aa95cc1ef21-Abstract-Conference.html.

[20] Chien Van NGUYEN et al. « A Survey on Small Language Models ». In : *Proceedings of the 15th International Conference on Recent Advances in Natural Language Processing – Natural Language Processing in the Generative AI Era*. Shoumen, Bulgaria : INCOMA Ltd., 2025, p. 807-821. DOI : 10.26615/978-954-452-098-4-093.

[21] Sihyeong PARK et al. « A Survey on Inference Engines for Large Language Models : Perspectives on Optimization and Efficiency ». In : *ACM Transactions on Intelligent Systems and Technology* (2026). DOI : 10.1145/3803798.

[22] Zheyu SHEN et al. « EdgeLoRA : An Efficient Multi-Tenant LLM Serving System on Edge Devices ». In : *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. MobiSys '25. Hilton Anaheim, Anaheim, CA, USA : Association for Computing Machinery, 2025, p. 138-153. ISBN : 9798400714535. DOI : 10.1145/3711875.3729141. URL : https://doi.org/10.1145/3711875.3729141.

[23] Ashish VASWANI et al. « Attention Is All You Need ». In : *Advances in Neural Information Processing Systems*. T. 30. 2017.

[24] Jie YE et al. « Characterizing the Behavior and Impact of KV Caching on Transformer Inferences Under Concurrency ». In : *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 2025, p. 1191-1202. DOI : 10.1109/IPDPS64566.2025.00108.

[25] Gyeong-In YU et al. « Orca : A Distributed Serving System for Transformer-Based Generative Models ». In : *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 2022, p. 521-538.

[26] Gyeong-In YU et al. « Orca : A Distributed Serving System for Transformer-Based Generative Models ». In : *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA : USENIX Association, juill. 2022, p. 521-538. ISBN : 978-1-939133-28-1. URL : https://www.usenix.org/conference/osdi22/presentation/yu.

[27] Lianmin ZHENG et al. « SGLang : Efficient Execution of Structured Language Model Programs ». In : *Advances in Neural Information Processing Systems*. Sous la dir. d'A. GLOBERSON et al. T. 37. Curran Associates, Inc., 2024, p. 62557-62583. DOI : 10.52202/079017-2000. URL : https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf.

[28] Yue ZHENG et al. « A Review on Edge Large Language Models : Design, Execution, and Applications ». In : *ACM Comput. Surv.* 57.8 (mars 2025). ISSN : 0360-0300. DOI : 10.1145/3719664. URL : https://doi.org/10.1145/3719664.

[29] Yue ZHENG et al. « A Review on Edge Large Language Models : Design, Execution, and Applications ». In : *ACM Computing Surveys* 57.8 (2025), p. 1-35. DOI : 10.1145/3719664.

[30] Pan ZHOU et al. « SLICE : SLO-Driven Scheduling for LLM Inference on Edge Computing Devices ». In : *IEEE Transactions on Services Computing* 19.4 (2026), p. 3072-3084. DOI : 10.1109/TSC.2026.3699152.